
Aplicacions Distribuïdes

Silvia Llorente
Jaime Delgado

Distributed Multimedia Applications Group
Departament d'Arquitectura de Computadors

(*) Some slides come from other sources

HTTP based applications and services development

- Web applications
 - Development, tools and applications types: jsp, servlet, python, static, HTML5, etc.
- Programming tools
 - IDE: Netbeans, Eclipse, IntelliJ, other
 - Application servers

HTTP based applications and services development

- Web services
 - SOAP: WSDL, SOAP message format, operation invocation, result collection, operation definition, etc.
 - REST: A different way of implementing web services

HTTP based applications and services development

- Distributed applications for mobile devices
 - Mobile devices features
 - Some examples of development

Web applications

- Applications that will be accessed through the Internet by a browser
- The ones developed in class
 - Based on the Servlet and Java Server Pages specification originally defined by Sun, now owned by Oracle
 - Other: asp Microsoft, Python, php, scripts cgi, javascript, etc.

Static Web Applications

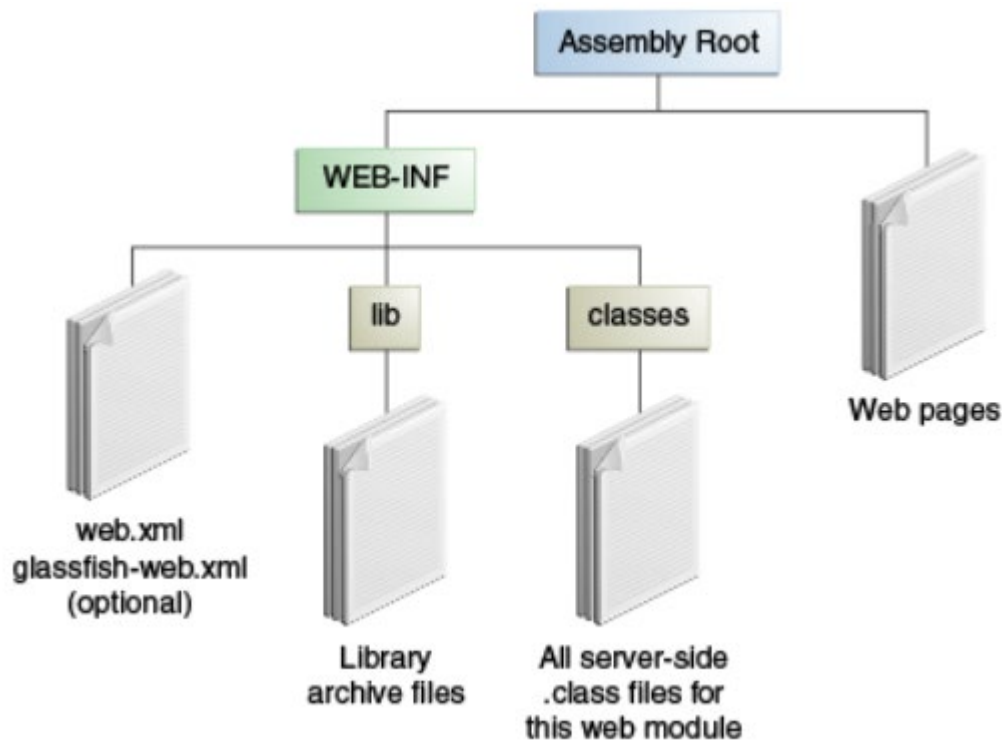
- HTML pages
- Images
- Static resources
 - Text files
 - Pdf files
 - Other static files ...
- Elements do not change over time
 - Good to cache them (proxy, browser, etc.)

Dynamic Web Applications

- Generate HTML code using some programming language
 - Oracle: Java Server Pages, Servlets
 - Microsoft: ASP, C#
 - Javascript
 - PHP
 - Executables (.exe, scripts, etc.)
 - Python
 - Any software able to generate HTML code
- Elements can be dynamically generated
 - Database access, files and external services, etc.
 - Also static elements (images, documents, etc.)

Deployed J2EE web applications structure

- Web pages folder
 - Static HTML pages
 - Java Server Pages
 - Images or other public files (pdf, doc's, etc.)
 - It may have subfolders



Web applications structure

Source: <https://javaee.github.io/tutorial/packaging003.html>

Structure (continued)

- WEB-INF folder
 - Special directory of the web application, which contains configuration information
 - Classes
 - Contains java classes (.class) that uses the application
 - Lib
 - Contains java libraries (.jar) that uses the application
 - web.xml
 - Web application configuration file
 - In latest J2EE versions Java Annotations are used

Reference: <https://docs.oracle.com/javase/tutorial/java/annotations/basics.html>

web.xml example file (J2EE 9)

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee  
    https://jakarta.ee/xml/ns/jakartaee/web-app 5 0.xsd"  
  version="5.0">
```

XML Schema

```
<session-config>  
  <session-timeout>  
    30  
  </session-timeout>  
</session-config>
```

```
</web-app>
```

Reference: <https://jakarta.ee/specifications/servlet/5.0/jakarta-servlet-spec-5.0.pdf>

Servlets

- Servlet
 - Class dynamically loaded in the server to add functionality
 - Execution in a Java Virtual Machine (JVM)
 - Platform independent (!!)
 - How it responds to the client?
 - Generates HTML code
 - Redirects to another page
 - jsp, static html, another servlet, etc.

Servlet lifecycle

- 1) Create and initialize servlet
- 2) Process web clients requests
- 3) Destroy the servlet, freeing memory and resources (*garbage collection*)

API Servlets

- 2 java packages
 - javax.servlet (new jakarta.servlet)
 - javax.servlet.http (new jakarta.servlet.http)
- Generic servlets (javax.servlet, jakarta.servlet)
 - Do not have **main()**
 - Invoke **service()** method when receiving a request

HTTP Servlets

- Specific for HTTP
- **service()** method already implemented
 - Invokes appropriate HTTP method
 - doXxx (where Xxx can be Get, Post, other HTTP methods)

Java Server Pages (JSP)

- Mix HTML and Java code
- They are especially used to build interfaces (not command processing)
- Special tags to integrate HTML code with Java code (`<% %>`)

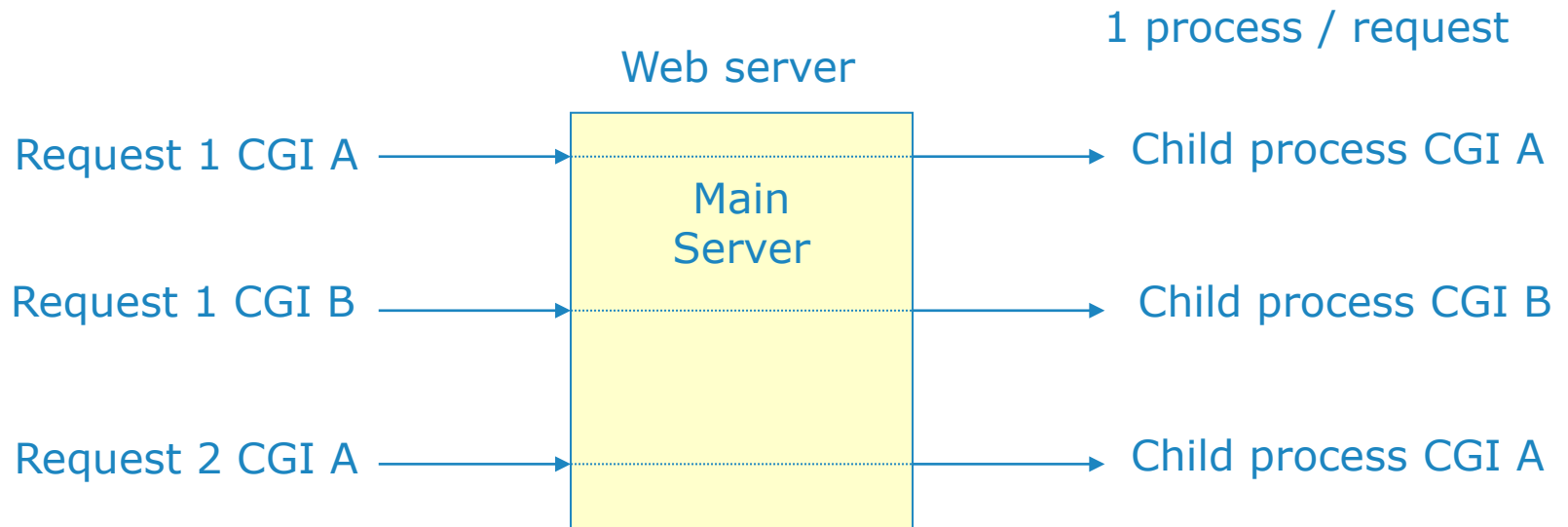
Servlets vs Java Server Pages

- Similar functionality
- Jsp, easier installation and development
 - Only the user interface, more complex to process
- Server internally converts JSP into a servlet
 - Generates Java code to build the response
 - Everything that is HTML code is generated and sent to the client using Java code
 - Compiles the code
 - Invokes this code when the jsp is called

Servlets vs Java Server Pages

- jsp compiled class is stored in a server internal folder
 - More difficult to debug
 - IDE's have improved and now servlets generated from jsp are easier to reach

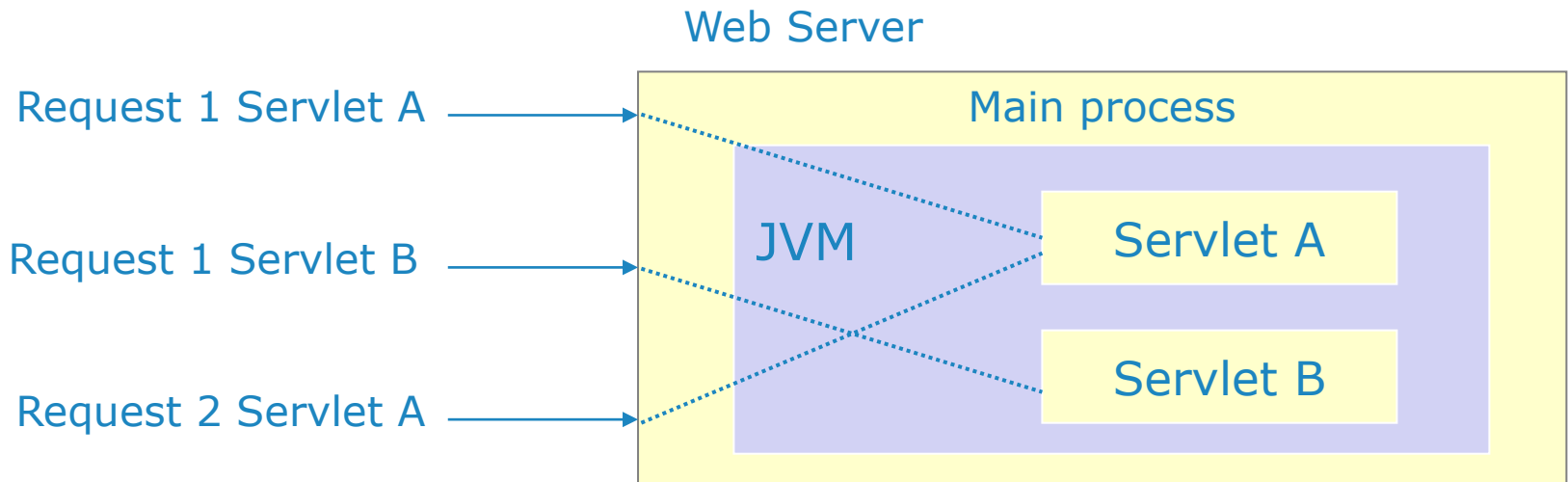
Execution environment: CGI example



CGI: Common Gateway Interface

Reference: http://www.oreilly.com/openbook/cgi/ch01_01.html

Execution environment: servlets example



Reference: <http://www.tutorialspoint.com/servlets/servlets-life-cycle.htm>

Application Servers

- The ones in the subject are based on the J2EE model
 - Originally defined by Sun, then Oracle, now passed to Apache and Eclipse
- They work both with web applications and web services

Application Servers

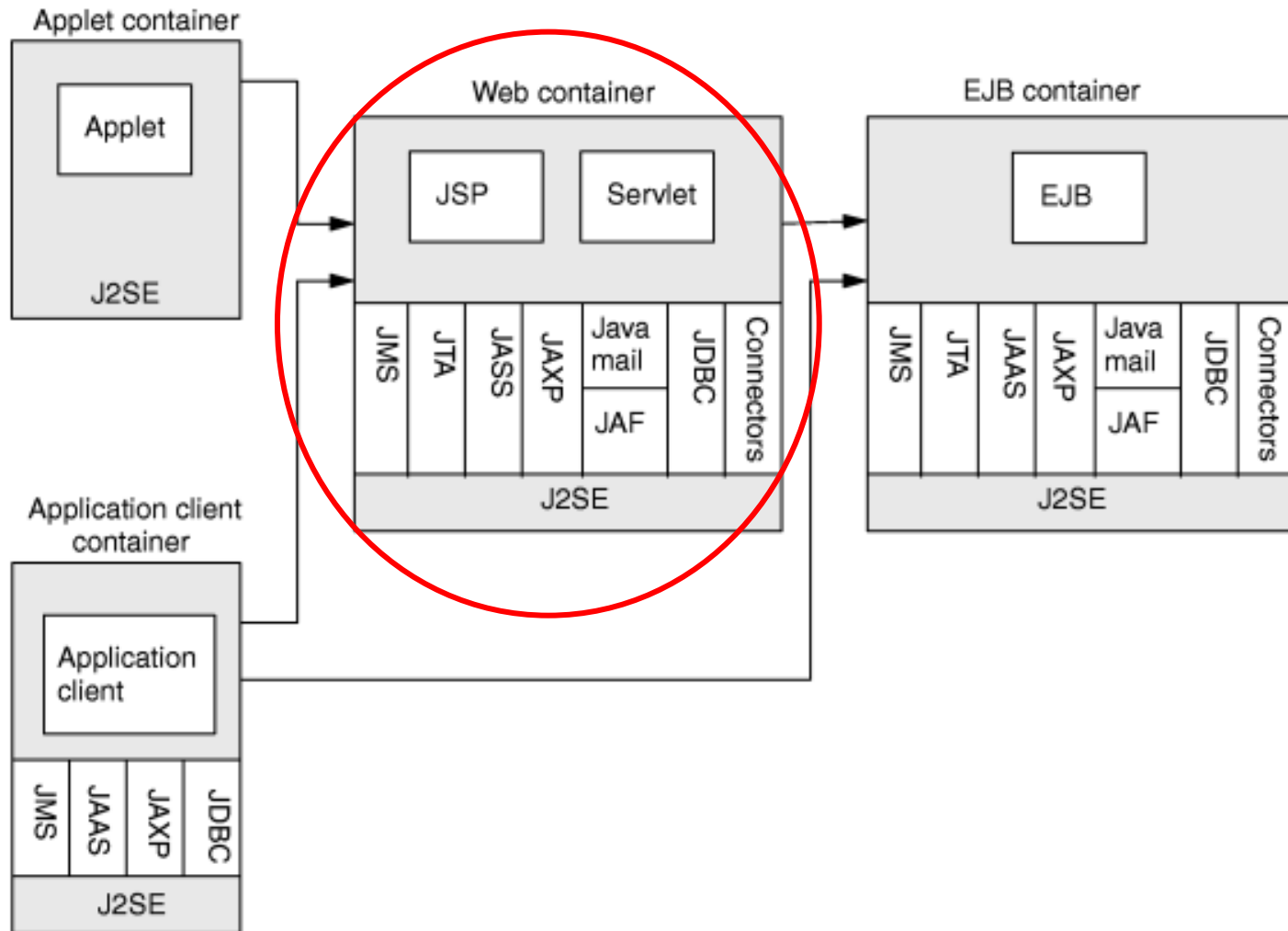
- They provide functionality to support the implementation of complex web applications and services (also used by companies)
- J2EE applications are portable between different application servers
 - Developed in Java language
 - Predefined folder structure
 - .war files, Web Application Archive

Reference: Compatible implementations with J2EE

<http://www.oracle.com/technetwork/java/javaee/overview/compatibility-jsp-136984.html>

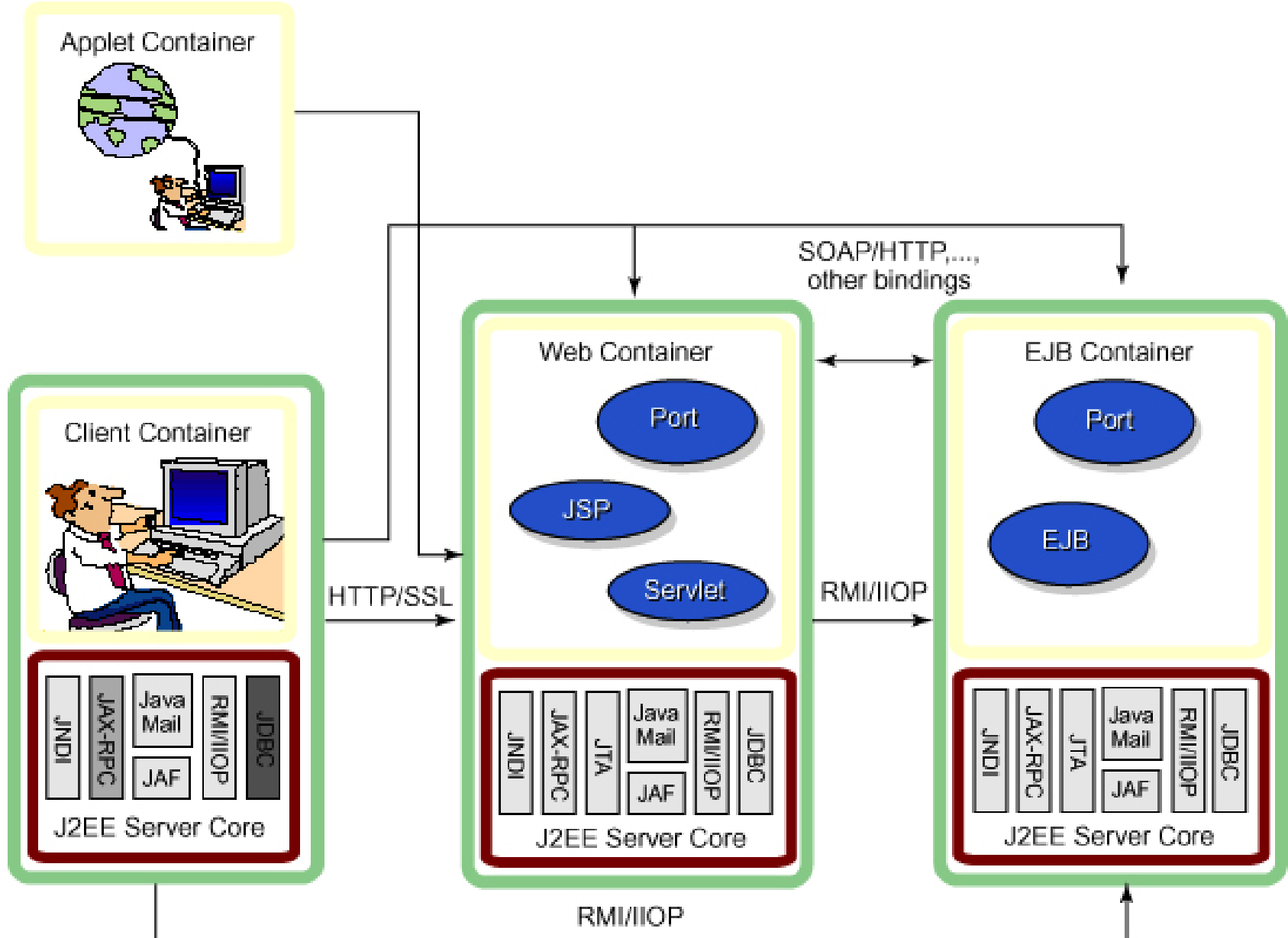
<https://jakarta.ee/compatibility/>

J2EE Application Server Architecture

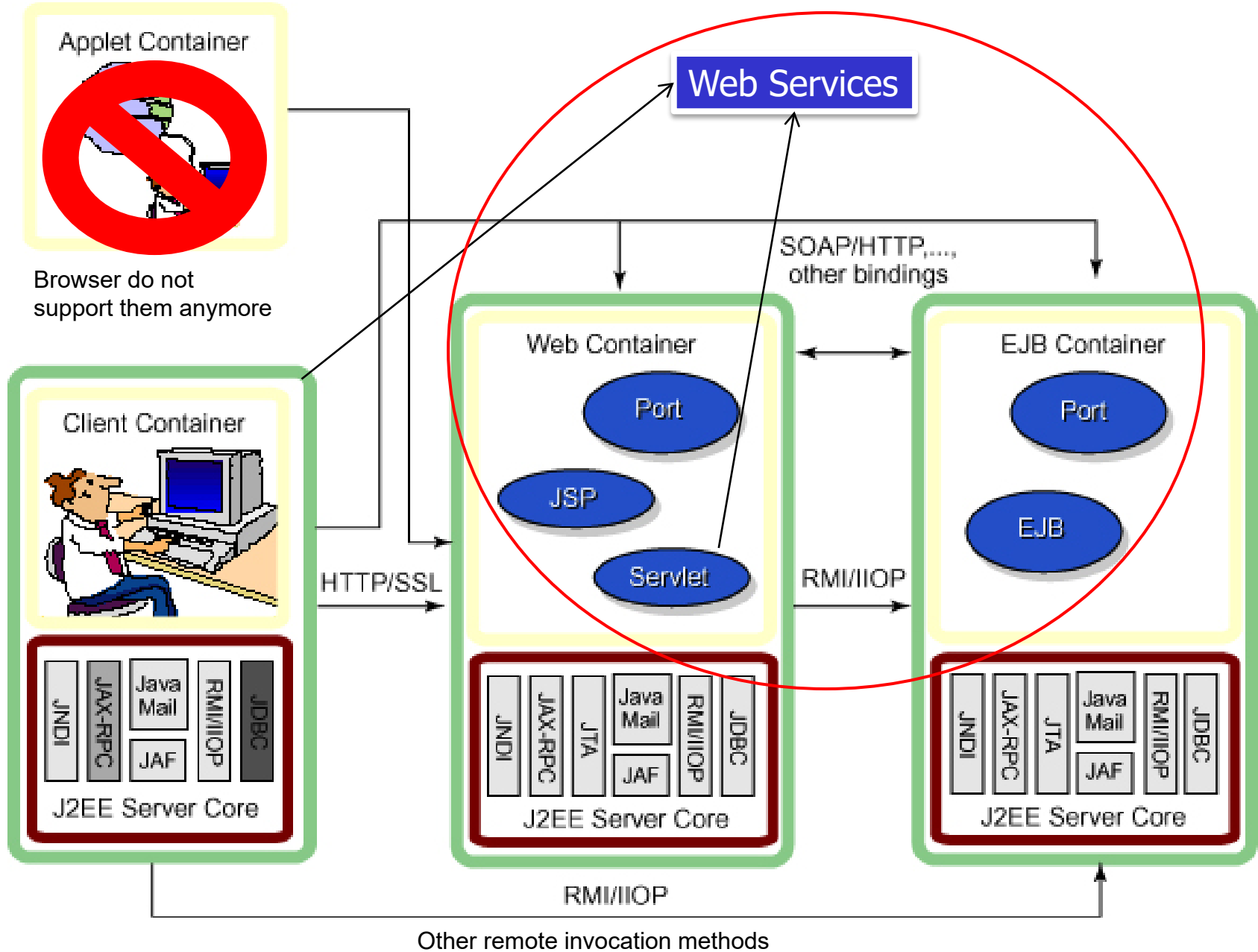


J2SE = Java 2 Platform, Standard Edition
JAAS = Java Authentication and Authorization Service
JAF = JavaBeans Activation Framework
JAXP = Java API for XML Parsing

JDBC = Java Database Connectivity
JMS = Java Message Service
JSP = Java Server Page
JTA = Java Transaction API



Reference: <https://www.semanticscholar.org/paper/J2EE-Architecture-and-Patterns-in-Enterprise-Guo/89e570fa9317c99e950219eeff1cb9091063af45>



Some tools

- J2EE application Servers
 - Glassfish
 - Tomcat
 - Payara
- IDE's
 - Apache Netbeans
 - IntelliJ
 - Eclipse

Web Services (WS)

- 2 approaches:
 - SOAP: Simple Object Access Protocol (still used, but not as before)
 - REST: REpresentational State Transfer

Reference: <https://blog.postman.com/soap-vs-rest/>

Common WS features

- Independent from
 - Platform
 - Programming language
- Work over HTTP(s)
- They can be freely used to pass through firewalls
 - Port 80 for http and 443 for https

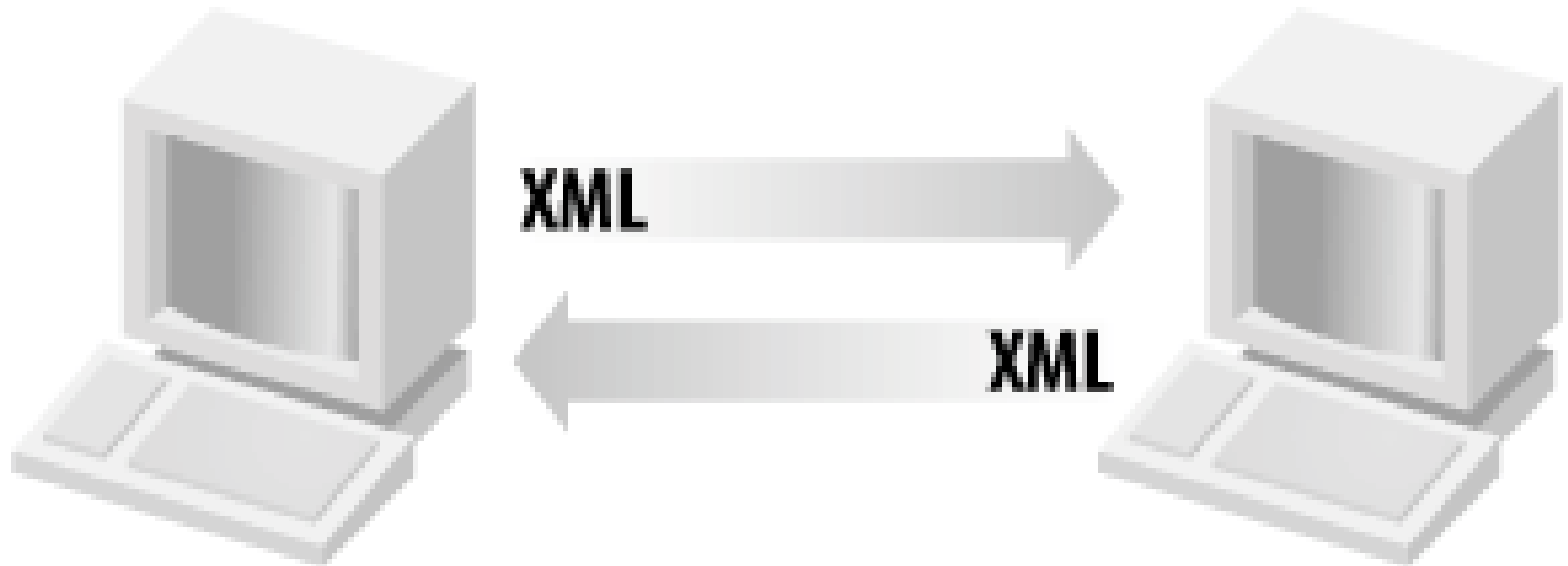
Common WS features (II)

- Do not provide: security, session management, quality of service
 - They can be added using user authentication tokens, Web Service Security, HTTPS (secure sockets)
 - We will see some examples in the security topic

WS

- Derived from RPC (Remote Procedure Call)
- Calls to remote methods through web using XML messages as data interchange language
- Objective:
 - Remote calls independent from
 - Programming language
 - Operating system where the service is installed

Basic WS



Computer A

Language: **C#**
Operating System: **Windows 11**

Computer B

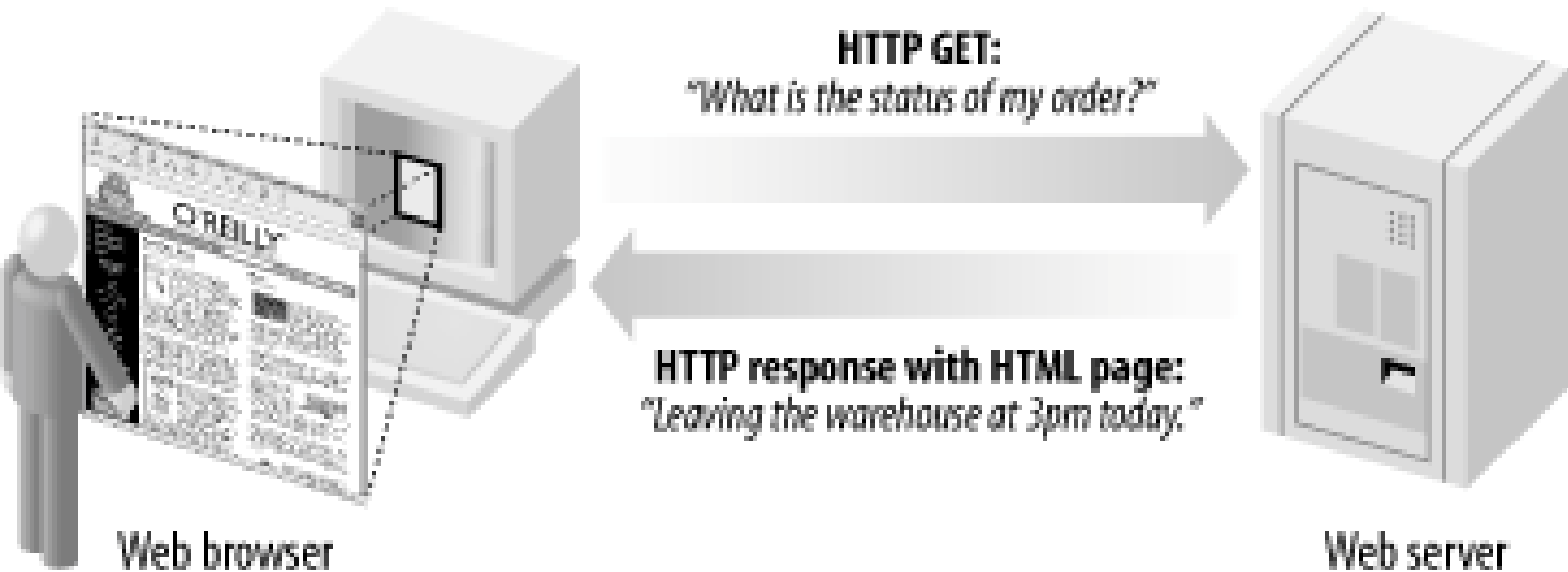
Language: **Java**
Operating System: **Linux**

WS with messages



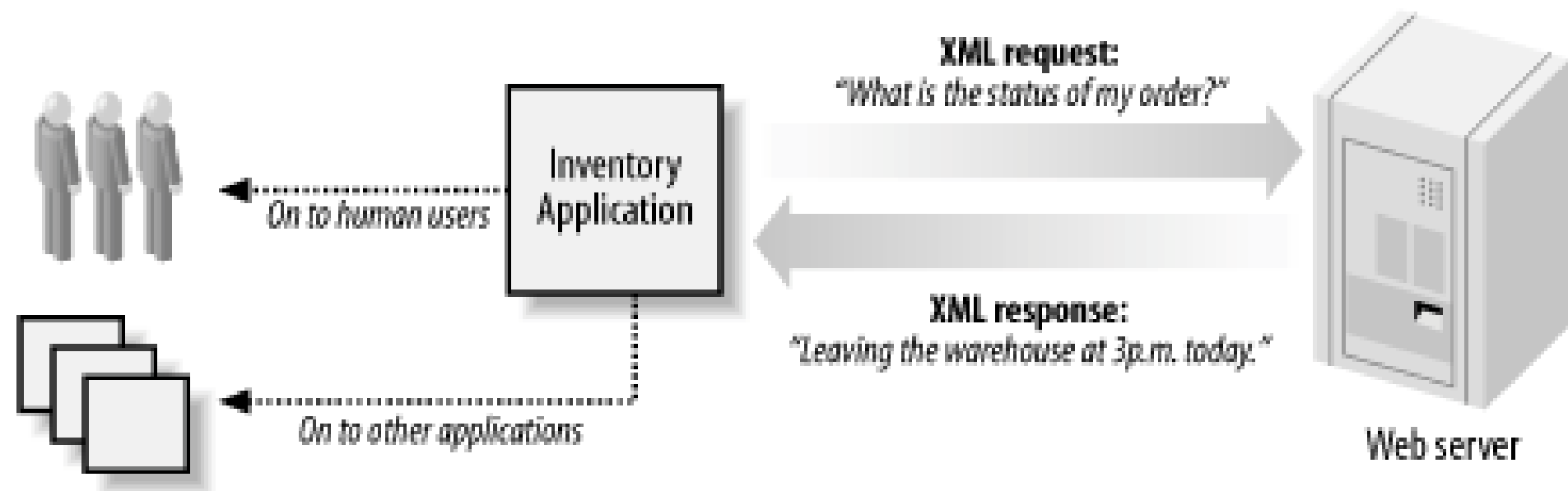
XML document

Web for the user



Reference: Web Services Essentials, Ethan Cerami, O'Reilly, ISBN: 9780596002244

Automated web



Reference: Web Services Essentials, Ethan Cerami, O'Reilly, ISBN: 9780596002244

Protocols and formats

WSDL

search and description

Universal Description, Discovery & Integration (UDDI)

XML

object method invocation

Simple Object Access Protocol (SOAP)

request-reply

Hypertext Transfer Protocol (HTTP)

transport

TCP/IP

Protocols and formats

WSDL

search and description

~~Universal Description, Discovery & Integration (UDDI)~~

XML

object method invocation

Simple Object Access Protocol (SOAP)

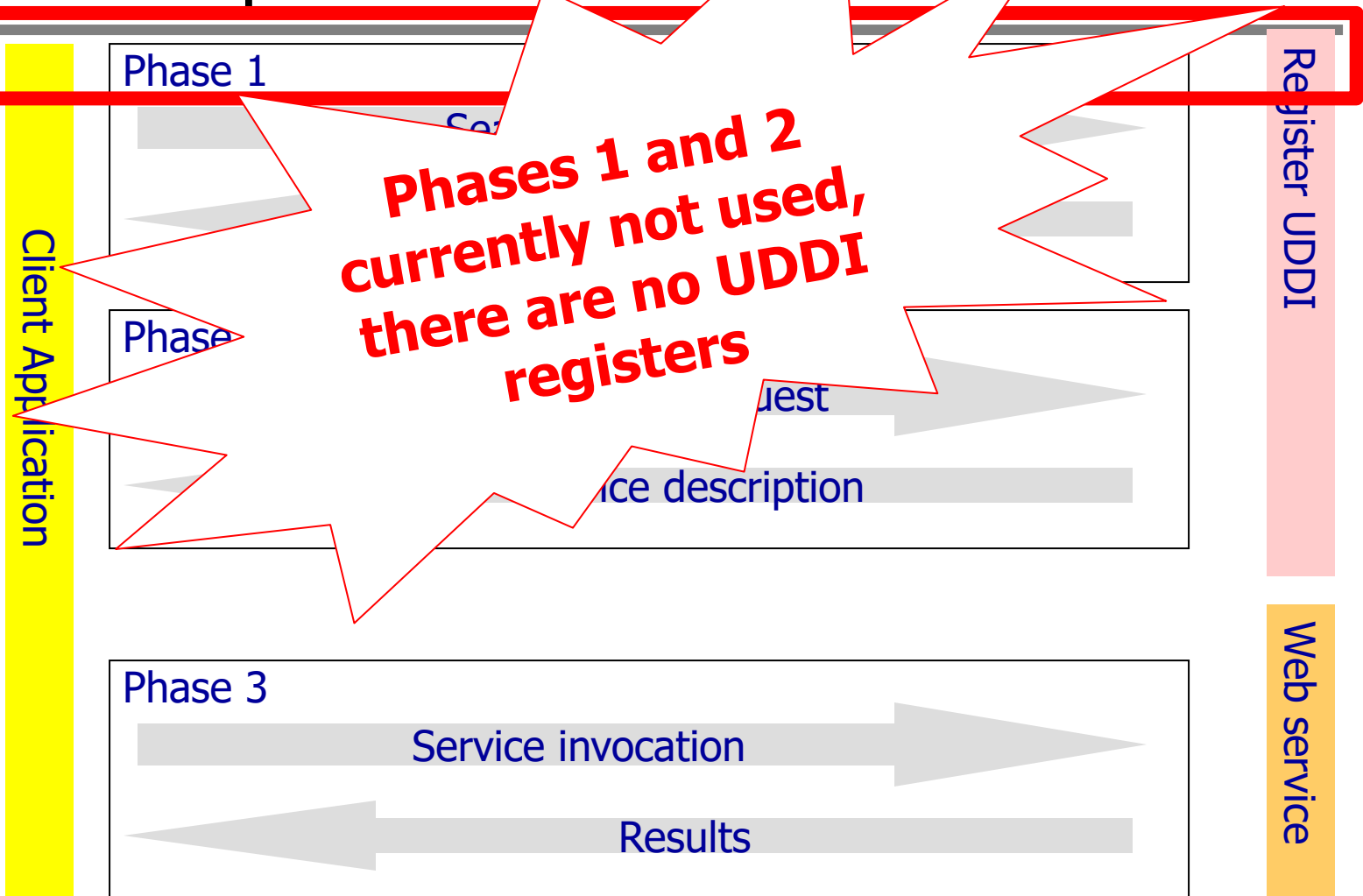
request-reply

Hypertext Transfer Protocol (HTTP)

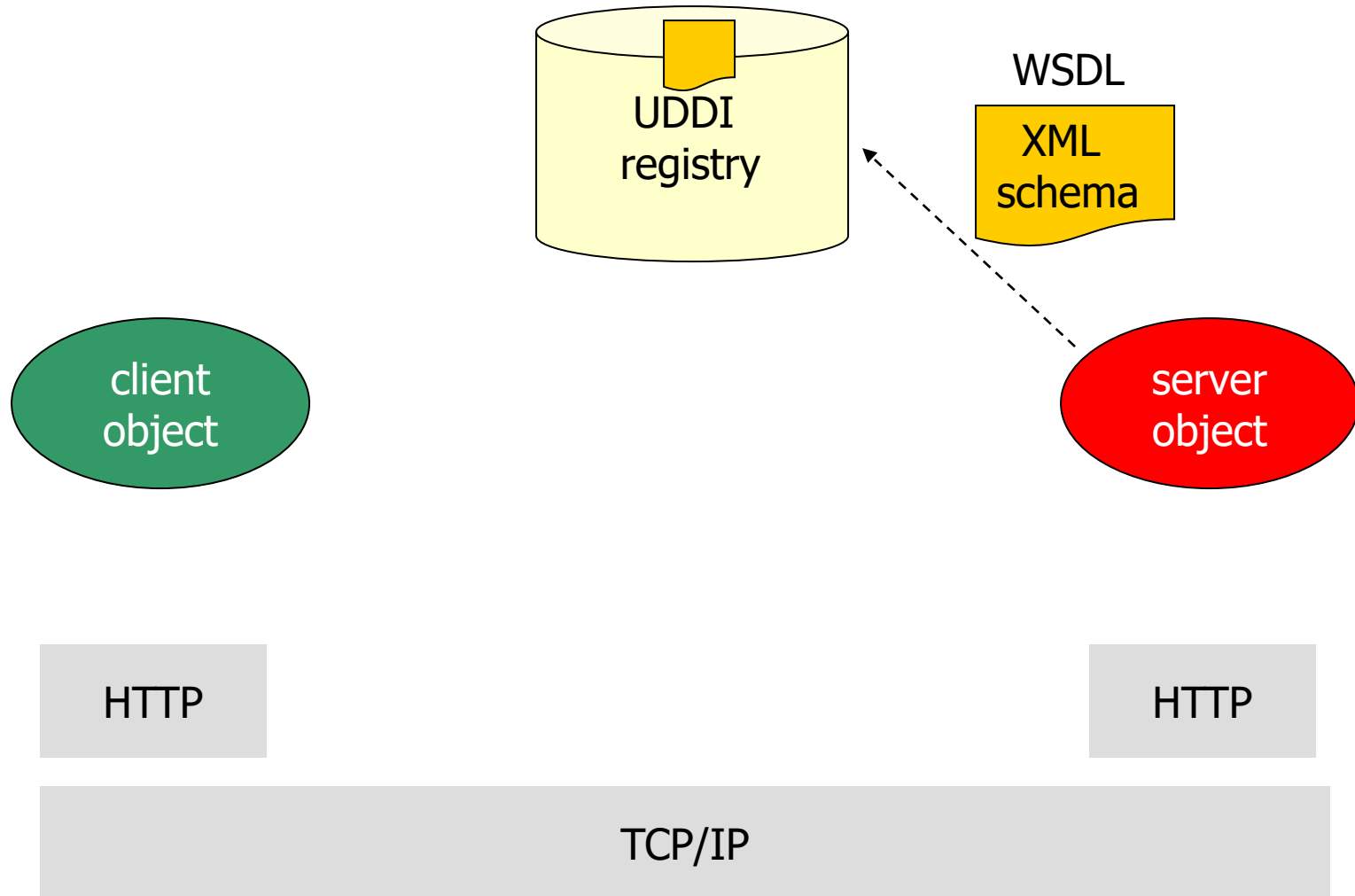
transport

TCP/IP

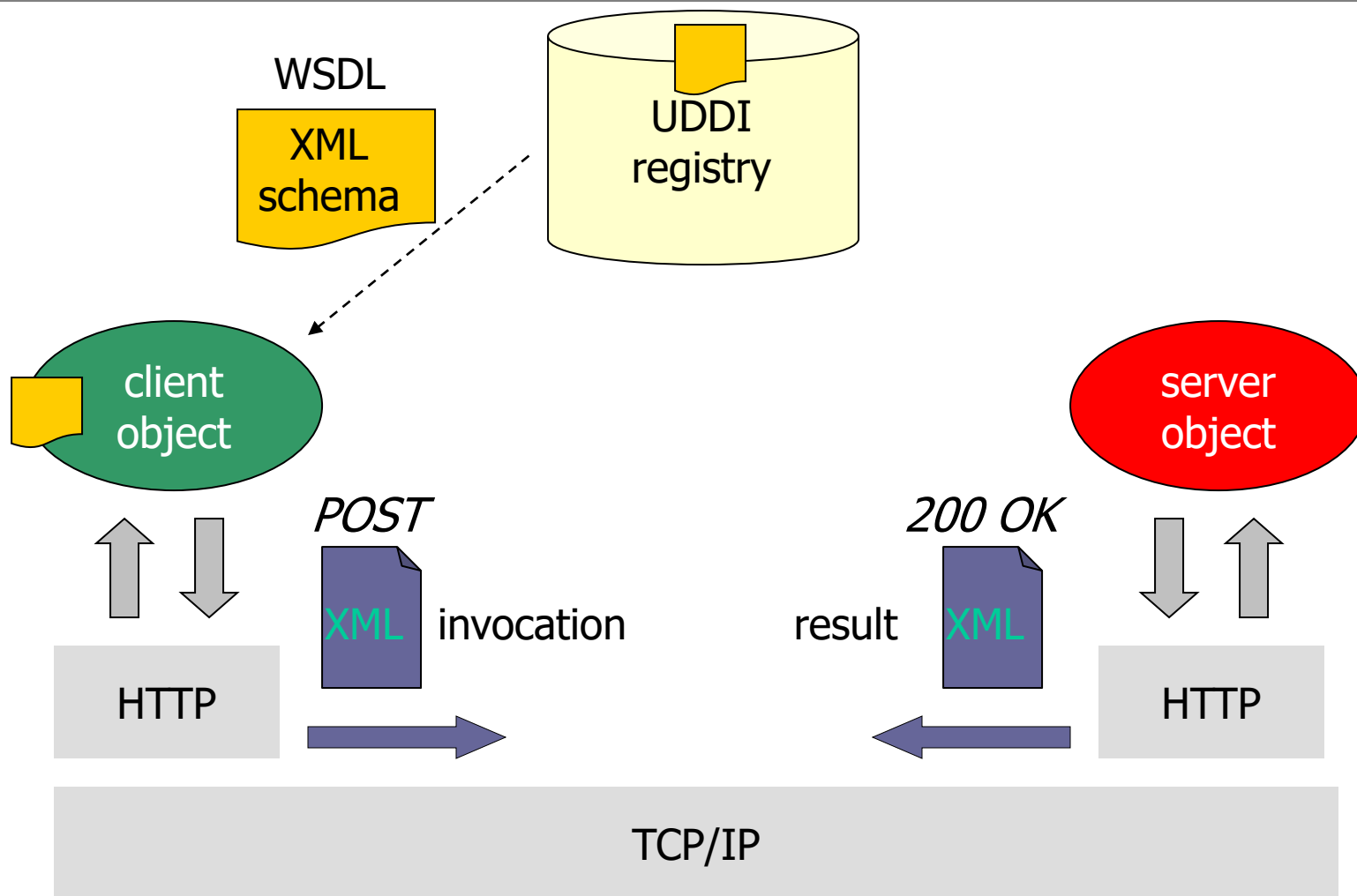
WS use phases



Remote method invocation with SOAP

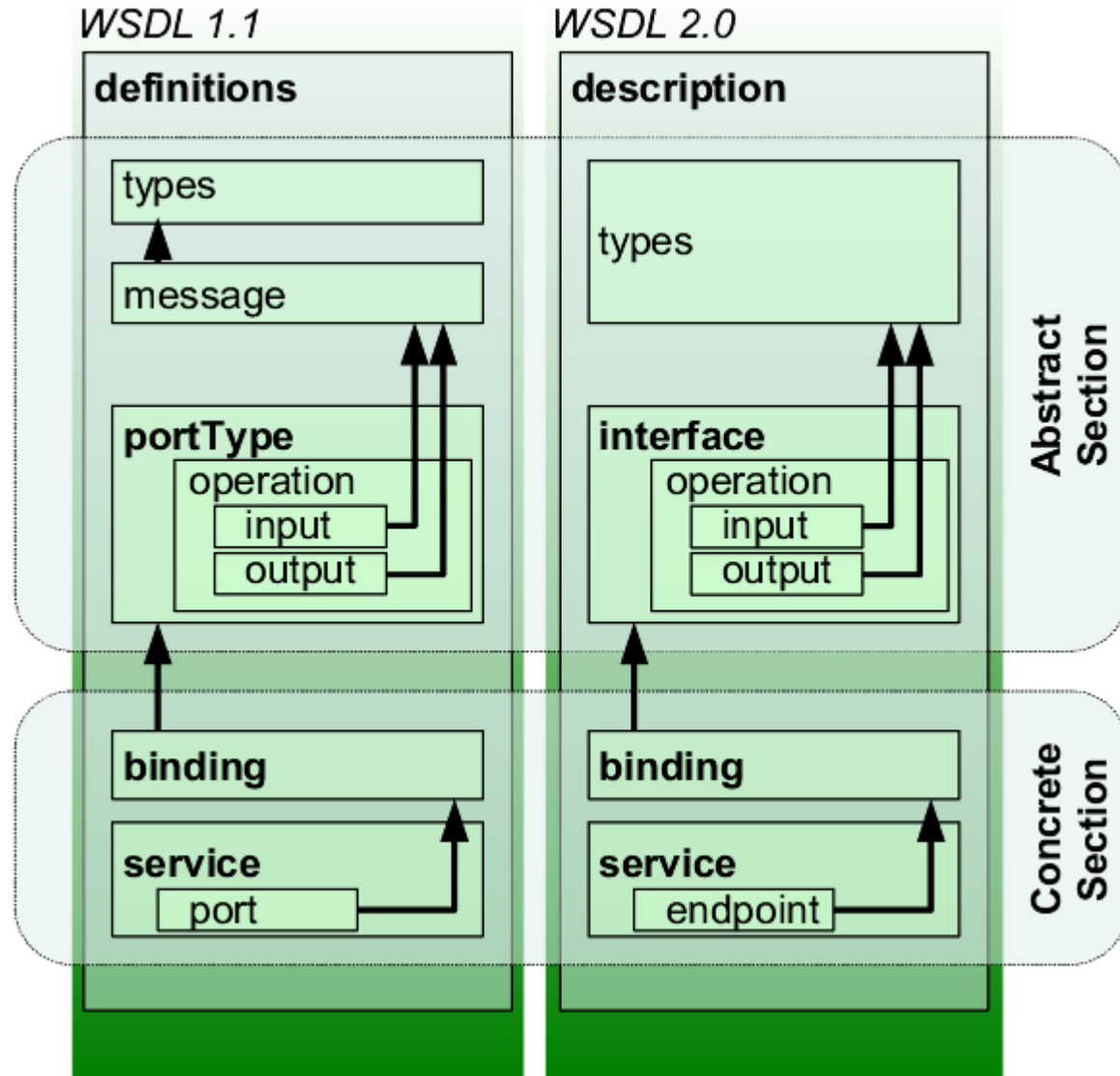


Remote method invocation with SOAP



WSDL

- Web Services **Description** Language (2.0)
 - SOAP y REST
- Web Services **Definition** Language (1.1)
 - SOAP



WSDL 1.1 example – SOAP Messages

```
<message name="getTermRequest">  
  <part name="term" type="xs:string"/>  
</message>
```

```
<message name="getTermResponse">  
  <part name="value" type="xs:string"/>  
</message>
```

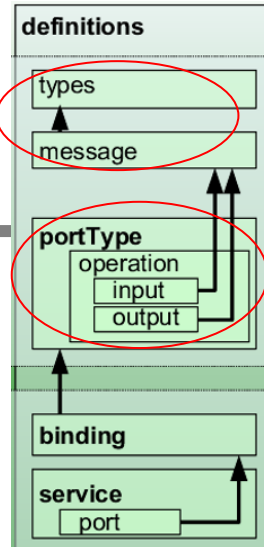
```
<portType name="glossaryTerms">  
  <operation name="getTerm">  
    <input message="getTermRequest"/>  
    <output message="getTermResponse"/>  
  </operation>  
</portType>
```

Request parameter

Result (response)

Remote method definition:

- Name
- Request message (input)
- Response message (output)



WSDL 1.1 example – SOAP Messages

```
<message name="getTermRequest">  
  <part name="term" type="xs:string"/>  
</message>
```

```
<message name="getTermResponse">  
  <part name="value" type="xs:string"/>  
</message>
```

```
<portType name="glossaryTerms">  
  <operation name="getTerm">  
    <input message="getTermRequest"/>  
    <output message="getTermResponse"/>  
  </operation>  
</portType>
```

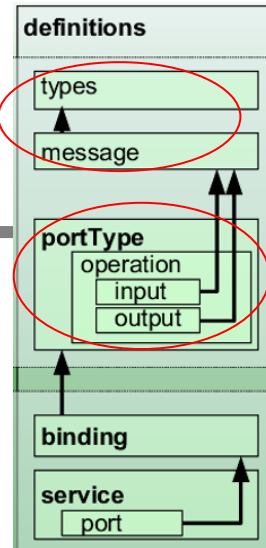
Request parameter

Result (response)

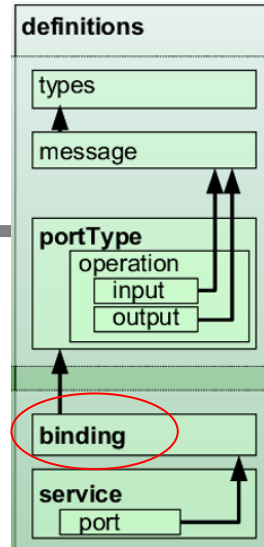
Remote method definition:

→ Name

- Request message (input)
- Response message (output)



WSDL 1.1 example – SOAP Binding



```
<message >  
</message>  
  
<portType>  
</portType>
```

To define other operations

```
<binding type="glossaryTerms" name="binding1">  
  <soap:binding style="document"  
    transport="http://schemas.xmlsoap.org/soap/http"/>  
  <operation>  
    <soap:operation  
      soapAction="http://example.com/getTerm"/>  
    <input><soap:body use="literal"/></input>  
    <output><soap:body use="literal"/></output>  
  </operation>  
</binding>
```

WSDL 1.1 – Complete SOAP example

WSDL root element, definitions

```
<?xml version="1.0"?>
```

```
<definitions name="StockQuote"
```

```
targetNamespace="http://example.com/stockquote.wsdl"
```

```
xmlns:tns="http://example.com/stockquote.wsdl"
```

```
xmlns:xsd1="http://example.com/stockquote.xsd"
```

```
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
xmlns="http://schemas.xmlsoap.org/wsdl/">
```

...

Namespaces that point to wsdl and xsd files define the operations and types of the web service

tns → Target namespace (the file that is being defined)

xsd1 → XML schema with the data types defined in this service

Reference: <https://cs.au.dk/~amoeller/WWW/webservices/wsdlexample.html>

WSDL 1.1 – Complete SOAP example

```
<?xml version="1.0"?>
```

```
<definitions name="StockQuote"
```

```
  targetNamespace= "http://example.com/stockquote.wsdl"
```

```
    xmlns:tns= "http://example.com/stockquote.wsdl"
```

```
    xmlns:xsd1="http://example.com/stockquote.xsd"
```

```
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
    xmlns= "http://schemas.xmlsoap.org/wsdl/">
```

...

Other namespaces that we use in this file

soap → SOAP elements XML schema

xmlns → WSDL XML schema. As it does not have a prefix we can directly use its elements

WSDL 1.1 – Complete SOAP example

```
<types>
```

```
  <schema targetNamespace="http://example.com/stockquote.xsd"
    xmlns="http://www.w3.org/2000/10/XMLSchema">
```

```
    <element name="TradePriceRequest">
```

```
      <complexType>
```

```
        <all>
```

```
          <element name="tickerSymbol" type="string"/>
```

```
        </all>
```

```
      </complexType>
```

```
    </element>
```

```
    <element name="TradePrice">
```

```
      <complexType>
```

```
        <all>
```

```
          <element name="price" type="float"/>
```

```
        </all>
```

```
      </complexType>
```

```
    </element>
```

```
  </schema>
```

```
</types>
```

Request

Type definition. It
may be in a different
file, xsd type

Response

WSDL 1.1 – Complete SOAP example

```
<message name="GetLastTradePriceInput">  
    <part name="body" element="xsd1:TradePriceRequest"/>  
</message>  
  
<message name="GetLastTradePriceOutput">  
    <part name="body" element="xsd1:TradePrice"/>  
</message>
```

Request / response messages

WSDL 1.1 – Complete SOAP example

```
<portType name="StockQuotePortType">  
  <operation name="GetLastTradePrice">  
    <input message="tns:GetLastTradePriceInput"/>  
    <output message="tns:GetLastTradePriceOutput"/>  
  </operation>  
</portType>
```

SOAP operation

WSDL 1.1 – Complete SOAP example

```
<binding name="StockQuoteSoapBinding" type="tns:StockQuotePortType">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="GetLastTradePrice">
      <soap:operation
        soapAction="http://example.com/GetLastTradePrice"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
```

HTTP Binding

Document/literal style

WSDL styles

They define how data is sent in request and response SOAP

Example operation: `public void myMethod (int x, float y);`

With Document/literal:

```
<soap:envelope>
  <soap:body>
    <xElement>5</xElement>
    <yElement>5.0</yElement>
  </soap:body>
</soap:envelope>
```

There is not operation or type

With Document/literal wrapped:

```
<soap:envelope>
  <soap:body>
    <myMethod>
      <x>5</x>
      <y>5.0</y>
    </myMethod>
  </soap:body>
</soap:envelope>
```

A *wrapper* has to be defined for the operation parameters

WsdI styles reference:

<https://www.ibm.com/docs/en/baw/24.x?topic=files-wsdl-binding-styles>

WSDL 1.1 – Complete SOAP example

```
<service name="StockQuoteService">
  <documentation>My first service</documentation>
  <port name="StockQuotePort" binding="tns:StockQuoteBinding">
    <soap:address location="http://example.com/stockquote"/>
  </port>
</service>
```

```
</definitions>
```

Address where the service can be found

End of the document

SOAP - Structure

```
<?xml version="1.0"?>  
<soap:Envelope  
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"  
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
<soap:Header>  
  ...  
</soap:Header>
```

SOAP Header

```
<soap:Body>  
  ...  
</soap:Body>
```

SOAP Body

```
</soap:Envelope>
```

HTTP SOAP Request example

```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn
```

HTTP request

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-
encoding">
<soap:Body xmlns:m="http://www.example.org/stock">
  <m:StockPrice>
    <m:StockName>IBM</m:StockName>
  </m:StockPrice>
</soap:Body>
</soap:Envelope>
```

SOAP request

HTTP SOAP Response example

HTTP/1.1 200 OK HTTP response
Content-Type: application/soap+xml; charset=utf-8
Content-Length: nnn

<?xml version="1.0"?> SOAP response
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-
encoding">

<soap:Body xmlns:m="http://www.example.org/stock">
 <m:StockPriceResponse>
 <m:Price>34.5</m:Price>
 </m:StockPriceResponse>
</soap:Body>

</soap:Envelope>

SOAP analysis

Advantages

- Several *transport protocols* can be used
 - Mainly HTTP is used, but SMTP is also possible

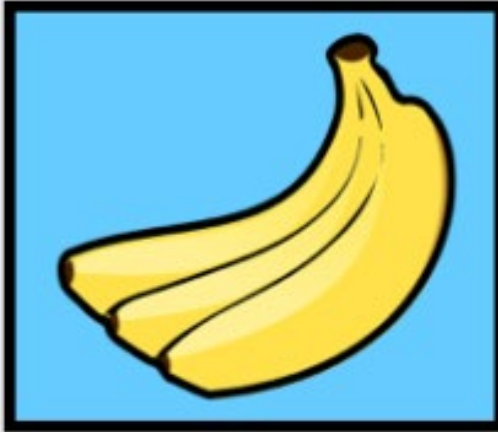
Disadvantages

- *Binding* by default SOAP + HTTP
 - Sends a lot of information → slow
- Not a problem for small messages
- Binary objects: Other mechanisms
 - *Message Transmission Optimization Mechanism* (MTOM) and *XML-binary Optimized Packaging* (XOP)

HTTP POST Message

SOAP Envelope

XML Data



XML Data

Since SOAP must be XML, then the image has to be converted into an XML datatype (xs:base64Binary) before it's sent. This process makes the data fat.

HTTP POST Message

SOAP Envelope

XML Data



XML Data

XOP tells receiver where to find the attachment

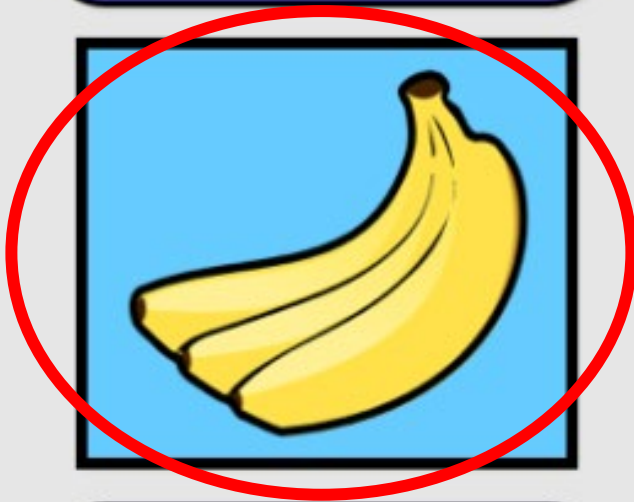


With MTOM, image will be transmitted outside the envelope as a MIME attachment - in short, it's sent according to its original datatype: a jpg, png, or gif.

HTTP POST Message

SOAP Envelope

XML Data



XML Data

Since SOAP must be XML, then the image has to be converted into an XML datatype (`xs:base64Binary`) before it's sent. This process makes the data fat.

HTTP POST Message

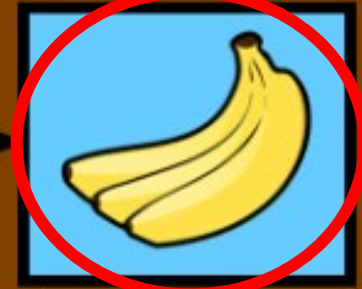
SOAP Envelope

XML Data

XML Data



XOP tells receiver where to find the attachment



With MTOM, image will be transmitted outside the envelope as a MIME attachment - in short, it's sent according to its original datatype: a jpg, png, or gif.

SOAP + MTOM + XOP

SOAP Message with a reference to the image

```
<soap:Body><soap:Envelope>  
<tns:data><xop:include href="SomeUniqueID-ThatLeadsToTheImage"/></tns:data>  
</soap:Body></soap:Envelope>
```

Content-id: "SomeUniqueID"

Content-Type: image/png

image binary data here

The image is sent in binary format, much less bytes than in Base64, increasing size in a 33%

SOAP + MTOM + XOP

SOAP Message with a reference to the image

```
<soap:Body><soap:Envelope>  
<tns:data><xop:include href="SomeUniqueID-ThatLeadsToTheImage"/></tns:data>  
</soap:Body></soap:Envelope>
```

Content-id: "SomeUniqueID"

Content-Type: image/png

image binary data here

The image is sent in binary format, much less bytes than in Base64, increasing size in a 33%

```
Content-Type: multipart/related; boundary=MIMEBoundary4A7AE55984E7438034;  
    type="application/xop+xml"; start="<0.09BC7F4BE2E4D3EF1B@apache.org>";  
    start-info="text/xml; charset=utf-8"
```

MIME Header

--MIMEBoundary4A7AE55984E7438034

```
content-type: application/xop+xml; charset=utf-8; type="application/soap+xml;" content-  
transfer-encoding: binary  
content-id: <0.09BC7F4BE2E4D3EF1B@apache.org>
```

SOAP Header

```
<?xml version='1.0' encoding='utf-8'?>  
<soapenv:Envelope xmlns:soapenv="...."....>  
    .....  
    <xop:Include href="cid:1.A91D6D2E3D7AC4D580@apache.org"  
        xmlns:xop="http://www.w3.org/2004/08/xop/include">  
    </xop:Include>  
    .....  
</soapenv:Envelope>
```

SOAP + XOP reference

--MIMEBoundary4A7AE55984E7438034

```
content-type: application/octet-stream  
content-transfer-encoding: binary  
content-id: <1.A91D6D2E3D7AC4D580@apache.org>
```

Binary content header

Binary Data.....

Binary Data

--MIMEBoundary4A7AE55984E7438034--

WSDL Examples

<http://www.w3.org/2001/03/14-annotated-WSDL-examples>

SOAP WS

- Java
- PHP
- .NET
- gSOAP
 - C, C++

- Any program supporting XML request and response messages can be a SOAP WS

REST: REpresentational State Transfer

- Simplified distributed applications architecture
- Uses HTTP methods to implement operations
 - No specific operations or methods are defined
- Not a standard
 - But based on them

SOAP vs. REST

- SOAP defines complex structures to make requests
 - REST does not require them
- Just an example...

Comparing SOAP and REST

- SOAP request example, sent with POST

```
<?xml version="1.0"?>
```

```
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
```

```
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
  <soap:body pb="http://www.acme.com/phonebook">
```

```
    <pb:GetUserDetails>
```

```
      <pb:UserID>12345</pb:UserID>
```

```
    </pb:GetUserDetails>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```


Comparing SOAP and REST

- REST request example, sent with GET

<http://www.acme.com/phonebook/UserDetails/12345>

Comparing SOAP and REST

- SOAP request example, sent with POST

```
<?xml version="1.0"?>
```

```
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
  <soap:body pb="http://www.acme.com/phonebook">
```

```
    <pb:GetUserDetails>
```

```
      <pb:UserID>12345</pb:UserID>
```

```
    </pb:GetUserDetails>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

- REST request example, sent with GET

```
http://www.acme.com/phonebook/UserDetails/12345
```

Equivalent request!

REST request example

Request:

GET /StockPrice/IBM HTTP/1.1

Host: example.org

Accept: text/xml

Accept-Charset: utf-8

REST response example

Response:

HTTP/1.1 200 OK

Content-Type: text/xml; charset=utf-8

Content-Length: nnn

```
<?xml version="1.0"?>
<m:Quote xmlns:m="http://www.example.org/stock">
  <m:StockName>IBM</m:StockName>
  <m:Price>34.5</m:Price>
</m:Quote>
```

XML can be used in REST. There is no limitation on response format (nor request)

REST requests

- Simple REST request, sent with GET
`http://www.acme.com/phonebook/UserDetails/12345`
- Complex REST request, also using GET
`http://www.acme.com/phonebook/UserDetails?firstName=John&lastName=Doe`

How could I send this information with POST?

CRUD operations with REST (HTTP)

- **Create**
 - HTTP PUT method (if it exists, it is updated)
- **Read**
 - HTTP GET method
- **Update**
 - HTTP PATCH method (for partial update)
- **Delete**
 - HTTP DELETE method
- **POST** is an alternative for each operation
 - i.e. when Read, if there are complex parameters, messages must go into the request / response message, not in the query line

REST responses

- REST response
 - Any processable format (XML, CSV, JSON – JavaScript Object Notation)
 - SOAP does not allow that, only XML
- HTML format
 - Not recommended as response message format, difficult to process
 - It can be done if the response destination is a person which sees the response in a browser

JSON vs. XML

```
{
  "menu": {
    "id": "file",
    "value": "File": {
      "popup": {
        "menuitem": [
          {
            "value": "New",
            "onclick": "CreateNewDoc()"
          },
          {
            "value": "Open",
            "onclick": "OpenDoc()"
          },
          {
            "value": "Close",
            "onclick": "CloseDoc()"
          }
        ]
      }
    }
  }
}
```

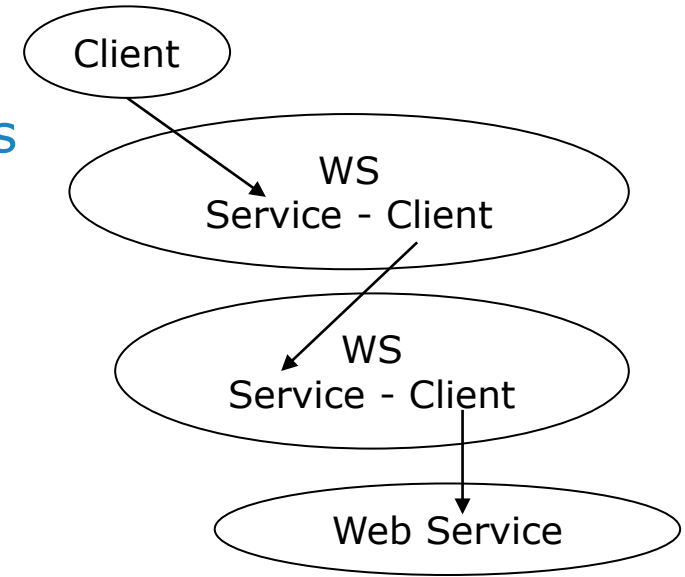
```
<menu id="file" value="File">
  <popup>
    <menuitem value="New" onclick="CreateNewDoc()" />
    <menuitem value="Open" onclick="OpenDoc()" />
    <menuitem value="Close" onclick="CloseDoc()" />
  </popup>
</menu>
```


REST components

- Client – Server System
 - A component may have both roles
 - Client in front of other services
 - Service responding requests
 - Multilevel architecture
- REST is stateless
 - Each request needs enough data to be responded independently
- Resources can be cached
 - Indicate when they expire
 - Use HTTP Cache headers

REST components

- Client – Server System
 - A component may have both roles
 - Client in front of other services
 - Service responding requests
 - Multilevel architecture
- REST is stateless
 - Each request needs enough data to be responded independently
- Resources can be cached
 - Indicate when they expire
 - Use HTTP Cache headers

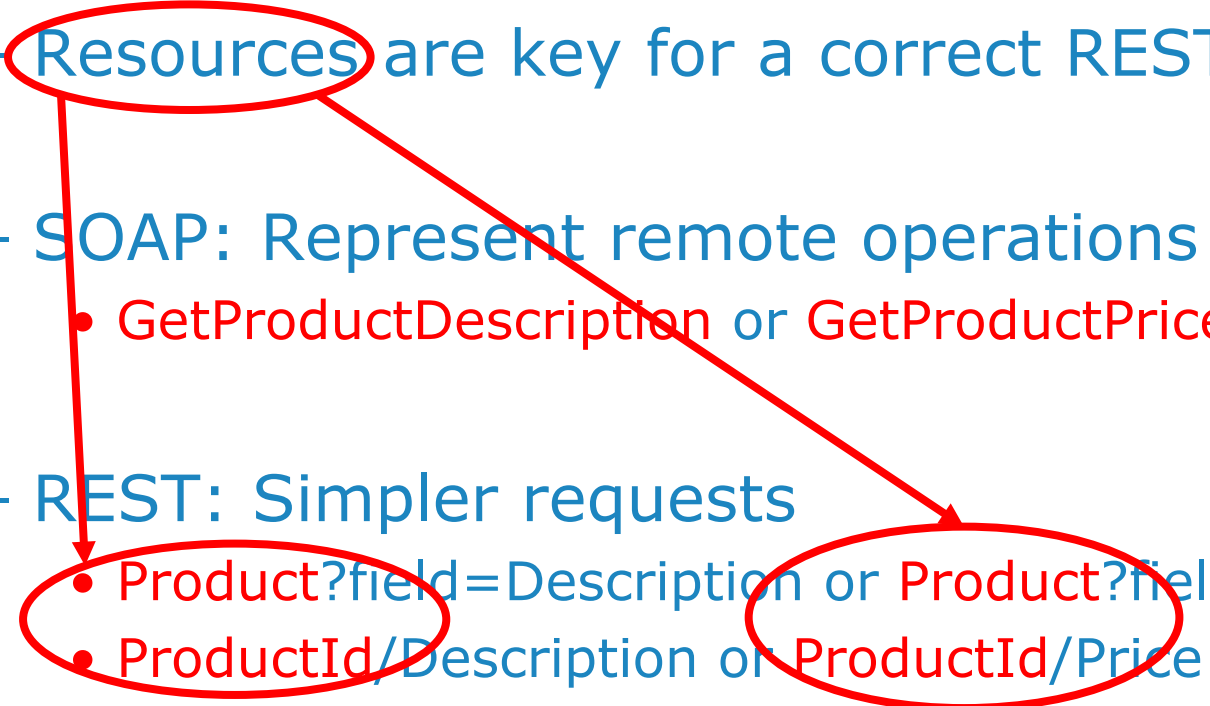


Multilevel architecture

REST components

- URL's identify resources
 - Universally accessible
 - Resources are key for a correct REST design
- SOAP: Represent remote operations
 - GetProductDescription or GetProductPrice
- REST: Simpler requests
 - Product?field=Description or Product?field=Price
 - ProductId/Description or ProductId/Price

REST components

- URL's identify resources
 - Universally accessible
 - Resources are key for a correct REST design
 - SOAP: Represent remote operations
 - GetProductDescription or GetProductPrice
 - REST: Simpler requests
 - Product?field=Description or Product?field=Price
 - ProductId/Description or ProductId/Price
- 

REST components

- Resource network

- Not complex resources

- Example: Products, accounts, users, etc.
 - Web user information account
web/UserId/AccountId

- Links to get extra info

- web/UserId/AccountId/movements

REST services design guide

- Use logical addresses, not physical
 - Using a logical address we can also return an xml or csv file
 - Example:
`http://www.server.com/restaurants/1/reservation`
 - Make a reservation in restaurant 1
 - This URL needs a physical file, but it could be a servlet, a php page, etc., that processes it
- A request does not have to return a lot of information, it is better to separate into blocks
 - Example: Show the first n products of a catalog and use the pagination concept

REST services design guide

- Response format should be documented
 - But not structured
 - It should not change, or clients would not understand the response!
 - Use API versions
- GET requests should **not** change the state of a resource
 - If resource has to be modified
 - POST or other HTTP methods like DELETE, PATCH or PUT should be used

Response codes for HTTP operations

Method	Correct response code	Error response code
POST	201 Created resource	404 Resource not found 409 Conflict, existing resource
GET	200 Correct	404 Resource not found
PUT	200 Correct 201 Created resource	204 No more content 404 Resource not found
PATCH	200 Correct	204 No more content 404 Resource not found
DELETE	200 Correct	404 Resource not found

Some real REST API's examples

- Instagram

- <https://developers.facebook.com/docs/instagram-basic-display-api/overview>

- Gmail

- <https://developers.google.com/gmail/api/reference/rest>

- API Racó de la FIB (authentication required)

- <https://api.fib.upc.edu/v2/>

Web services examples

- Gmail API usage
 - <https://developers.google.com/gmail/api/reference/rest/v1/users.messages/get>
- Instagram API usage
 - <https://developers.facebook.com/docs/instagram-basic-display-api/getting-started>

Some SOAP WS real examples

- [Flightxml.flightaware.com](http://flightxml.flightaware.com) – Flight information
 - <http://flightxml.flightaware.com/soap/FlightXML2/wsd1>
- Flickr, imàgenes
 - <https://www.flickr.com/services/api/request.soap.html>
 - <https://www.flickr.com/services/api/response.soap.html>

WS example - SOAP, WSDL

- FlightXML 2.0: Makes use of WSDL 2.0 to define SOAP service

```
<xs:element type="FlightXML2:AircraftTypeRequest" name="AircraftTypeRequest"/>
<xs:complexType name="AircraftTypeRequest">
  <xs:sequence>
    <xs:element type="xs:string" name="type" minOccurs="1" maxOccurs="1"/>
  </xs:sequence>
</xs:complexType>
```

Request

```
<xs:element type="FlightXML2:AircraftTypeResults" name="AircraftTypeResults"/>
<xs:complexType name="AircraftTypeResults">
  <xs:sequence>
    <xs:element type="FlightXML2:AircraftTypeStruct" name="AircraftTypeResult"/>
  </xs:sequence>
</xs:complexType>
<xs:element type="FlightXML2:AircraftTypeStruct" name="AircraftTypeStruct"/>
<xs:complexType name="AircraftTypeStruct">
  <xs:sequence>
    <xs:element type="xs:string" name="manufacturer" minOccurs="1" maxOccurs="1"/>
    <xs:element type="xs:string" name="type" minOccurs="1" maxOccurs="1"/>
    <xs:element type="xs:string" name="description" minOccurs="1" maxOccurs="1"/>
  </xs:sequence>
</xs:complexType>
```

Response

Documentation: <https://es.flightaware.com/commercial/flightxml/documentation2.rvt>
WSDL: <http://flightxml.flightaware.com/soap/FlightXML2/wsdl>

WS example - SOAP, WSDL

```
...
<wsdl:message name="AircraftTypeIn">
  <wsdl:part name="parameters" element="FlightXML2:AircraftTypeRequest"/>
</wsdl:message>
<wsdl:message name="AircraftTypeOut">
  <wsdl:part name="parameters" element="FlightXML2:AircraftTypeResults"/>
</wsdl:message>
```

Messages
Wrapped style

```
...
<wsdl:portType name="FlightXML2Soap">
  <wsdl:operation name="AircraftType">
    <wsdl:input message="FlightXML2:AircraftTypeIn"/>
    <wsdl:output message="FlightXML2:AircraftTypeOut"/>
  </wsdl:operation>
</wsdl:portType>
```

Operation
AircraftType

```
<wsdl:binding type="FlightXML2:FlightXML2Soap" name="FlightXML2Soap">
  <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="AircraftType">
    <soap:operation style="document" soapAction="FlightXML2:AircraftType"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
```

Binding
AircraftType

WS example - SOAP, WSDL

- FlightXML 2.0
 - Moving to REST
 - Still a good example

WS on-line example

- API Explorer for flickr:

<https://www.flickr.com/services/api/explore/flickr.test.echo>

- Several options:
 - XML over REST
 - JSONP
 - JSON
 - PHP serial (downloads a file)

Mobile devices: Application development

- Mobile devices
 - Introduction
 - Distributed applications and services for mobile devices

Introduction: Specific features

- Screen size
- Battery life
- (*Limited*) processing capabilities

Introduction: Device types

- Smartphones
- Mobile phones (not smartphones)
- Tablets
- eBook readers
- Smartwatches
- Other devices supporting mobile operating systems
 - Televisions, fridges, washing machines, induction cookers, thermostats, eHealth devices, etc.

Introduction: Development

- Mobile apps (200X)
 - Different (many of them) operating systems
 - Nokia – Symbian
 - Windows mobile
 - Blackberry
 - Palm
 - Ericsson, ...
 - App programming
 - Expensive
 - System dependent
 - Market fragmentation

Development (Cont.)

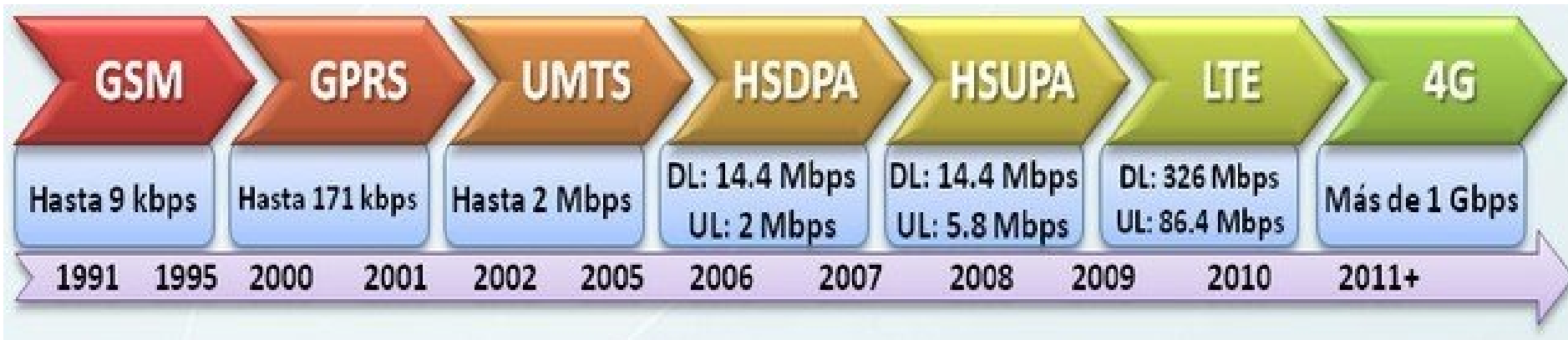
- Each system had a specific development environment
- Implementation of the same app for different mobile devices was not easy (different brands, different systems)
- Companies specialized on developing those kind of applications
 - Specific app for each brand and model
 - Required mobiles before market launch
 - They do not exist anymore, change in the development needs

Development (Cont.)

- JavaME (Java Micro Edition)
 - App development for different mobile devices
 - <https://www.oracle.com/java/technologies/javameoverview.html>
- Limited capabilities
 - In some cases, it was not possible to add new libraries
 - Example: Windows mobile vs Nokia

Development (Cont.)

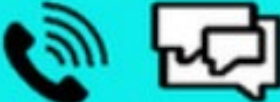
- Other development problems
 - Operators charged for GPRS downloads
 - No flat rate for internet connection from mobile devices
 - Limited download speed



Mobile networks coverage:

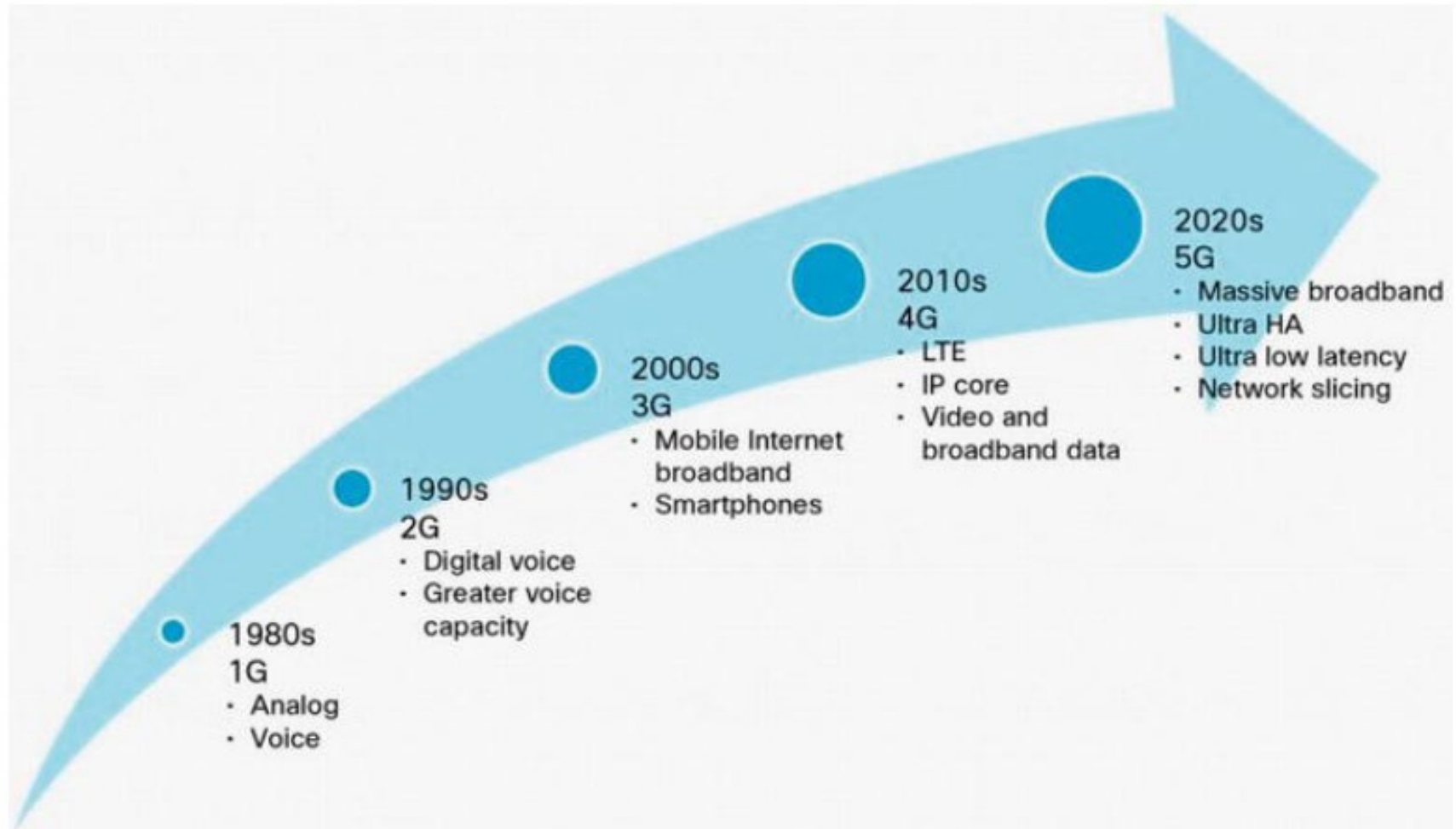
<https://www.nperf.com/en/map/ES/-/-/signal/?ll=36.12860325109547&lg=-6.92000000000000115&zoom=5>

Mobile networks speed

1G	2G	3G	4G	5G
				
				
1981	1992	2001	2010	2020
2 Kbps	64 Kbps	2 Mbps	100 Mbps	10 Gbps
Servicio básico de telefonía analógico	Servicio básico de telefonía digital (GSM) + mensajes de texto	Llega Internet al móvil	Banda ancha real (vídeo HD)	Internet de las cosas

Source: <https://es-us.vida-estilo.yahoo.com/5g-conexiones-r%C3%A1pidas-cobertura-211904180.html>

Mobile networks speed



Mobile networks acronyms

- GSM (Global System for Mobile Communications): Standard set developed by ETSI to describe protocols 2G digital cellular networks used by mobile phones.
- GPRS (General packet radio service): Is a packet oriented mobile data service on the 2G and 3G cellular communication system's
- EDGE (Enhanced Data rates for GSM Evolution): Also known as Enhanced GPRS (EGPRS) is a digital mobile phone technology that allows improved data transmission rates as a backward-compatible extension of GSM.
- UMTS (Universal Mobile Telecommunications System): Is a third generation mobile cellular system for networks based on the GSM standard.
- HSDPA (High-Speed Downlink Packet Access): Is an enhanced 3G (third-generation) mobile-telephony communications protocol in the High-Speed Packet Access (HSPA) family.

Mobile networks acronyms

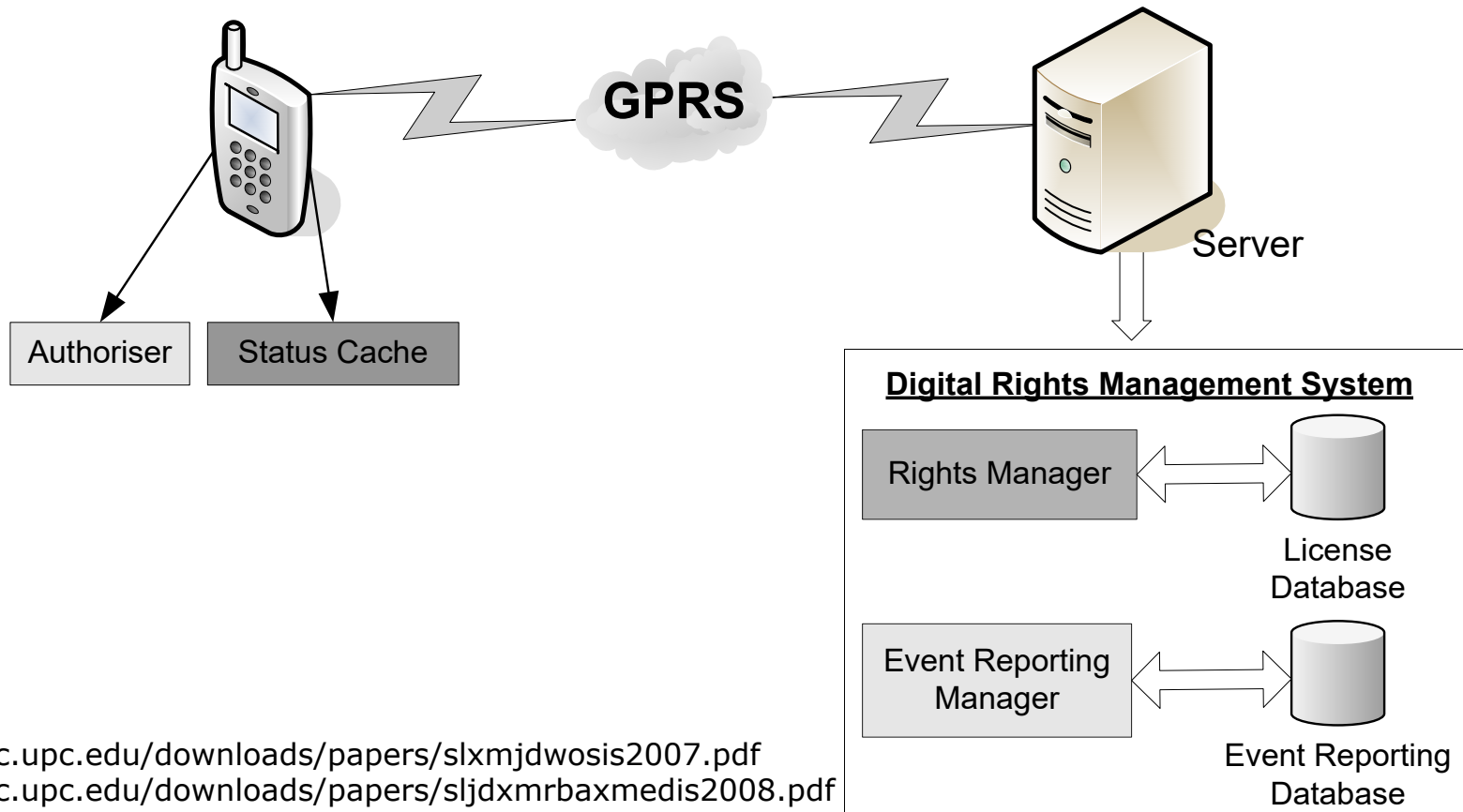
- HSUPA (High-Speed Uplink Packet Access): Is a 3G mobile telephony protocol in the HSPA family with up-link speeds up to 5.76 Mbit/s named by Nokia
- LTE (Long Term Evolution): Marketed as 4G LTE, is a standard for wireless communication of high-speed data for mobile phones and data terminals. It is based on the GSM/EDGE and UMTS/HSPA network technologies, increasing the capacity and speed using a different radio interface together with core network improvements.

What's next?

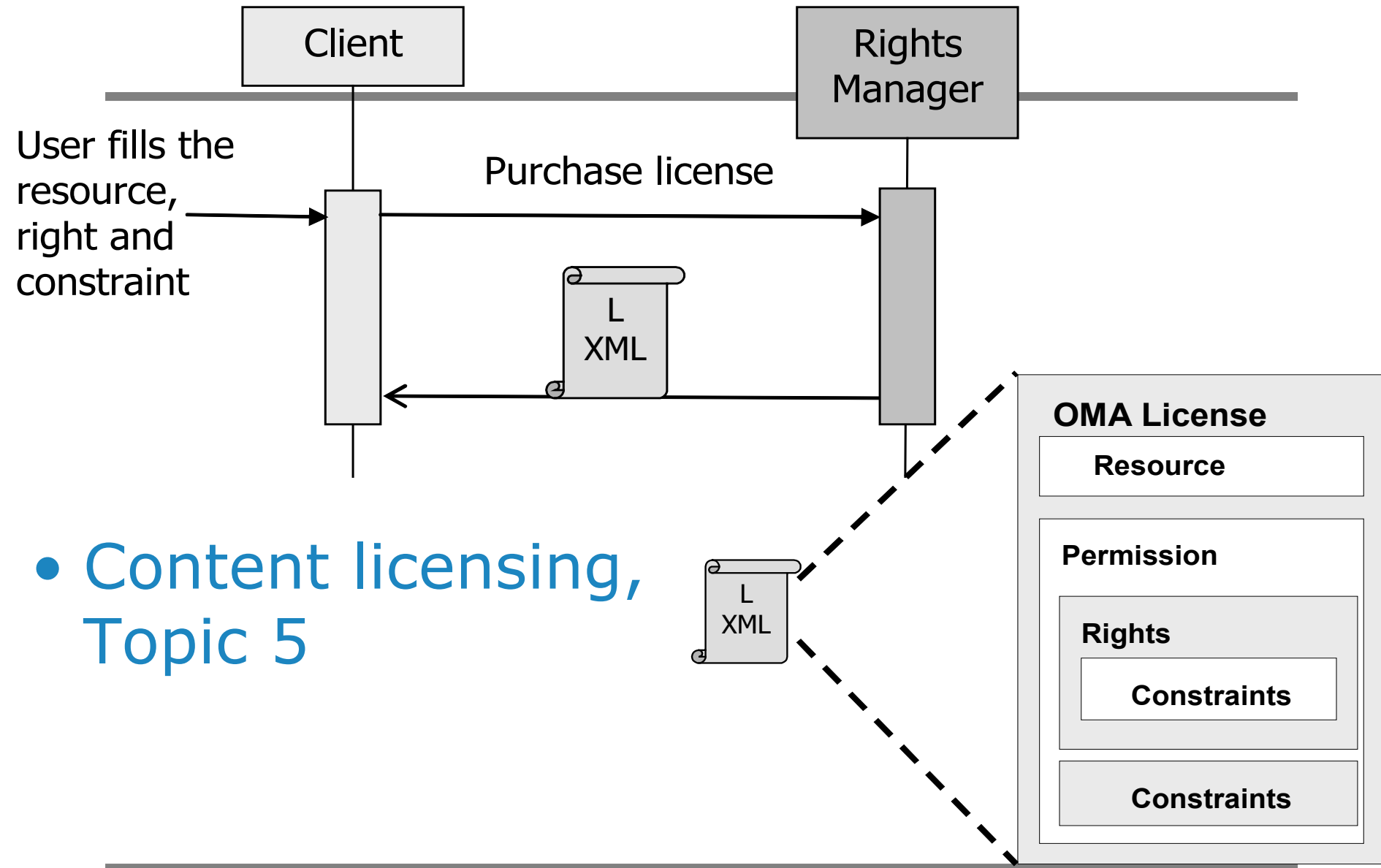
- 5G evolution and plans
 - <https://www.etsi.org/technologies/5G>
 - <https://blogthinkbig.com/5g-avanzado>
- 6G is coming...
 - <https://blogthinkbig.com/enable-6g>

Some distributed application examples

- Access control and event management



<http://dmag.ac.upc.edu/downloads/papers/slxmjdwoxis2007.pdf>
<http://dmag.ac.upc.edu/downloads/papers/sljdxmlrbaxmedis2008.pdf>



- Content licensing, Topic 5

```

<o-ex:rights xmlns:o-ex="http://odrl.net/1.1/ODRL-EX"
  xmlns:o-dd="http://odrl.net/1.1/ODRL-DD"
  xmlns:oma-dd="http://www.openmobilealliance.com/oma-dd"
  xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
  xmlns:xenc="http://www.w3.org/2001/04/xmlenc#">
  <o-ex:context>
    <o-dd:version>2.0</o-dd:version>
    <o-dd:uid>RightsObjectID</o-dd:uid>
  </o-ex:context>
  <o-ex:agreement>
    <o-ex:asset>
      <o-ex:context>
        <o-dd:uid>ContentID</o-dd:uid>
      </o-ex:context>
      <o-ex:digest>
        <ds:DigestMethod
          Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
        <ds:DigestValue>DCFHash</ds:DigestValue>
      </o-ex:digest>
      <ds:KeyInfo>
        <xenc:EncryptedKey>
          <xenc:EncryptionMethod
            Algorithm=
              "http://www.w3.org/2001/04/xmlenc#kw-aes128"/>
          <xenc:CipherData>
            <xenc:CipherValue>
              EncryptedCEK
            </xenc:CipherValue>
          </xenc:CipherData>
        </xenc:EncryptedKey>
        <ds:RetrievalMethod URI="REKReference"/>
      </ds:KeyInfo>
    </o-ex:asset>
    <o-ex:permission>
      <o-dd:play>
        <o-ex:constraint>
          <o-dd:datetime>
            <o-dd:end>2007-12-31T23:59:59Z</o-dd:end>
          </o-dd:datetime>
        </o-ex:constraint>
      </o-dd:play>
    </o-ex:permission>
  </o-ex:agreement>
</o-ex:rights>

```

```

<o-ex:rights xmlns:o-ex="http://odrl.net/1.1/ODRL-EX"
  xmlns:o-dd="http://odrl.net/1.1/ODRL-DD"
  xmlns:oma-dd="http://www.openmobilealliance.com/oma-dd"
  xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
  xmlns:xenc="http://www.w3.org/2001/04/xmlenc#">
  <o-ex:context>
    <o-dd:version>2.0</o-dd:version>
    <o-dd:uid>RightsObjectID</o-dd:uid>
  </o-ex:context>
  <o-ex:agreement>

```

```

    <o-ex:asset>
      <o-ex:context>
        <o-dd:uid>ContentID</o-dd:uid>
      </o-ex:context>
      <o-ex:digest>
        <ds:DigestMethod
          Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
        <ds:DigestValue>DCFHash</ds:DigestValue>
      </o-ex:digest>
      <ds:KeyInfo>
        <xenc:EncryptedKey>
          <xenc:EncryptionMethod
            Algorithm=
              "http://www.w3.org/2001/04/xmlenc#kw-aes128"/>
          <xenc:CipherData>
            <xenc:CipherValue>
              EncryptedCEK
            </xenc:CipherValue>
          </xenc:CipherData>
        </xenc:EncryptedKey>
        <ds:RetrievalMethod URI="REKReference"/>
      </ds:KeyInfo>
    </o-ex:asset>

```

Content

```

  <o-ex:permission>

```

```

    <o-dd:play>

```

Right

```

    <o-ex:constraint>

```

```

      <o-dd:datetime>
        <o-dd:end>2007-12-31T23:59:59Z</o-dd:end>
      </o-dd:datetime>

```

Conditions

```

    </o-ex:constraint>

```

```

  </o-dd:play>

```

```

</o-ex:permission>

```

```

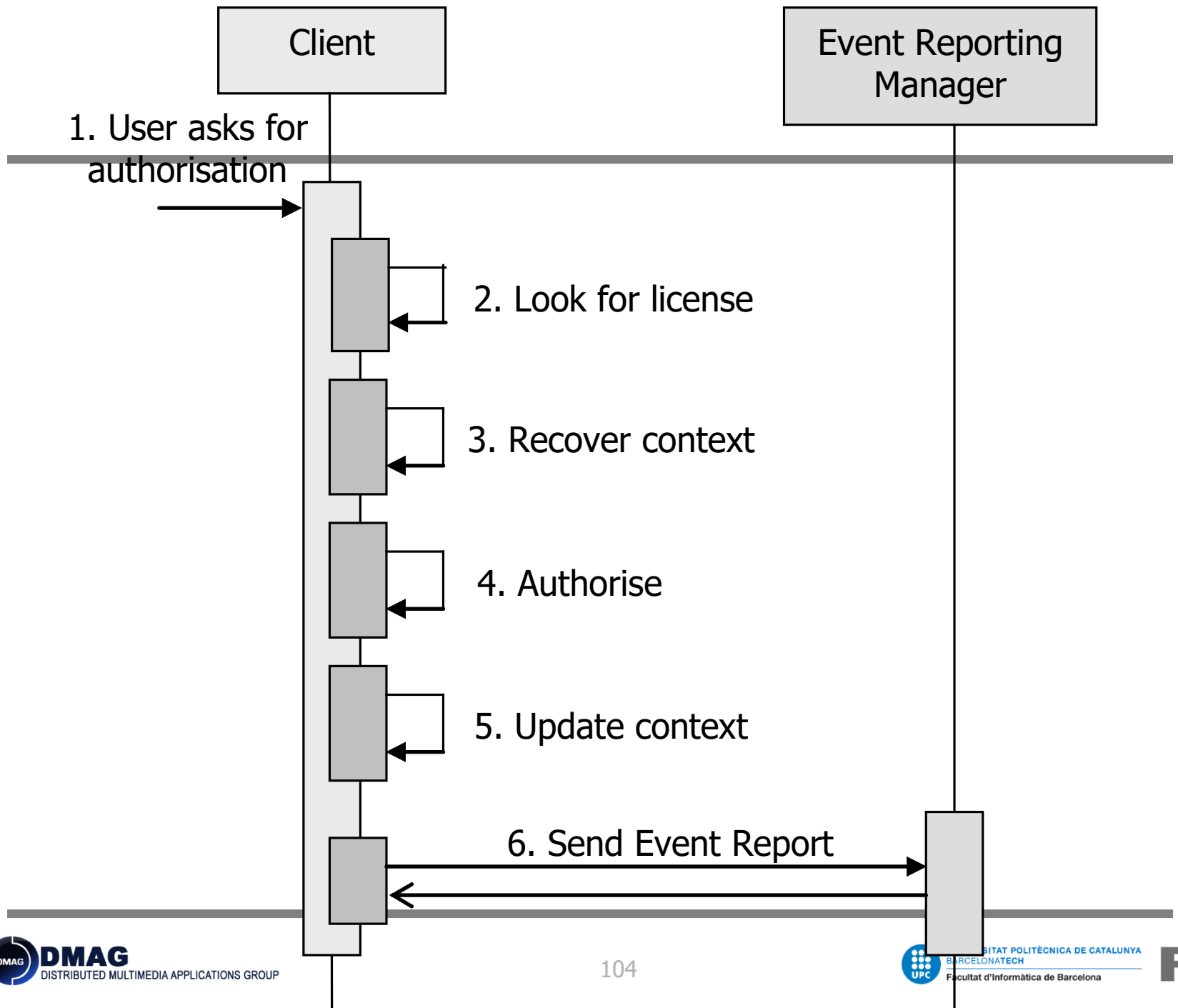
</o-ex:agreement>

```

```

</o-ex:rights>

```



Developed App

OMA Creator Client -

Time constraint license

Execution constraint license

Authorisation

Look at cache

Time constraint license

Resource

resource

Right ♦ Display

Time constraint

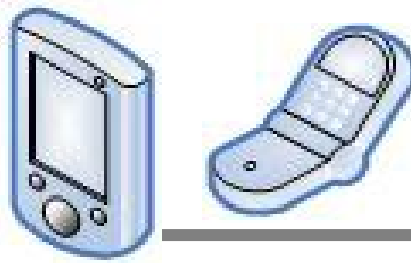
- ☒ A minute
- ☐ An hour
- ☐ A day

📶

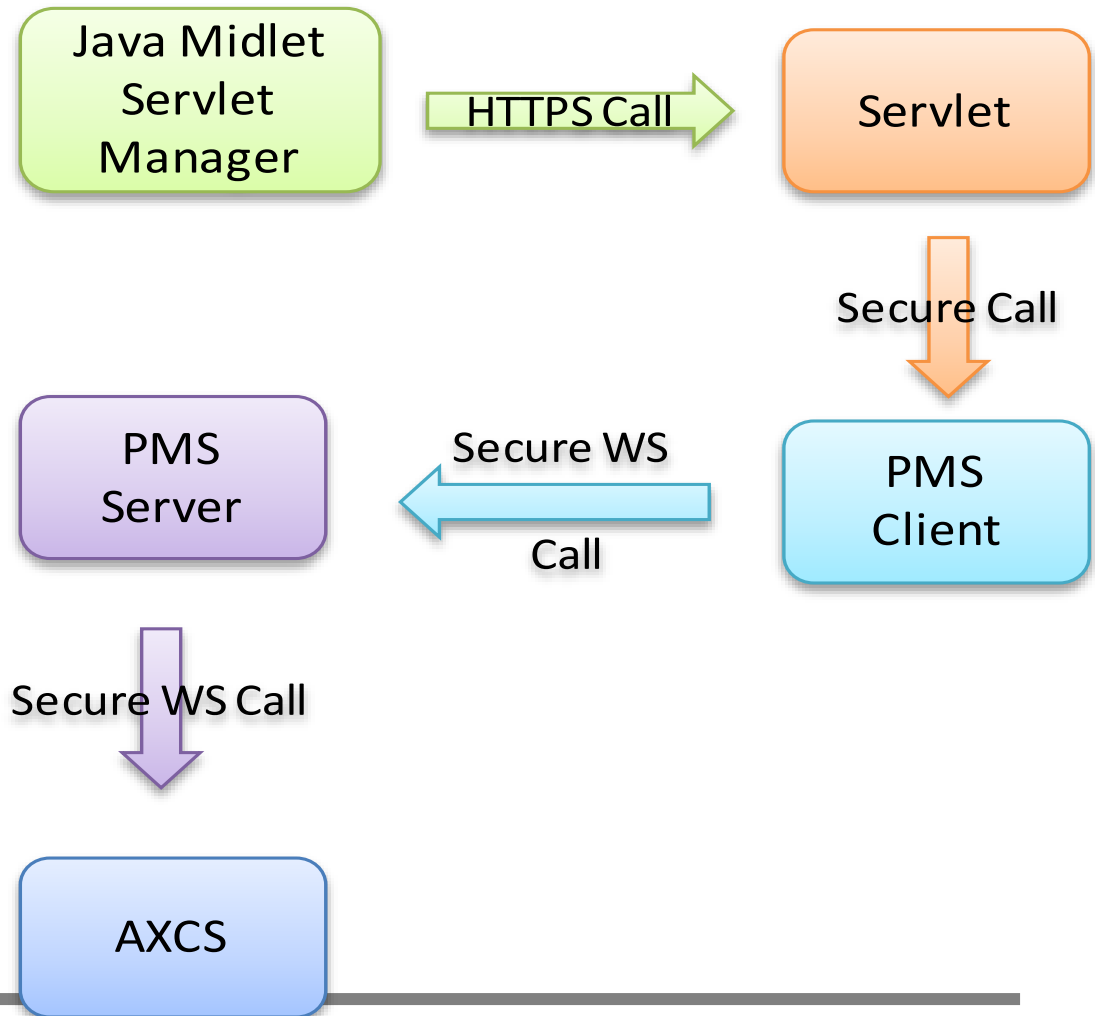
Buy result

Successful buy

License ID LID44



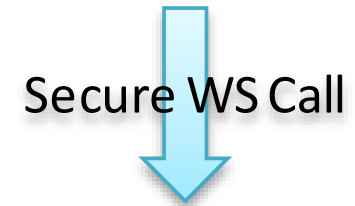
- Access control to the content using mobile WS
- Limited app, need of an intermediate servlet



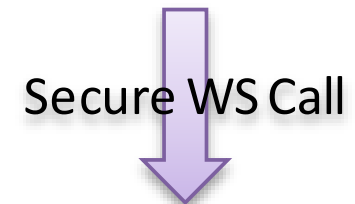
- Access control to the content using mobile WS smartphone
- Direct WS call
- Java libraries could be added to the device



PMS
Client

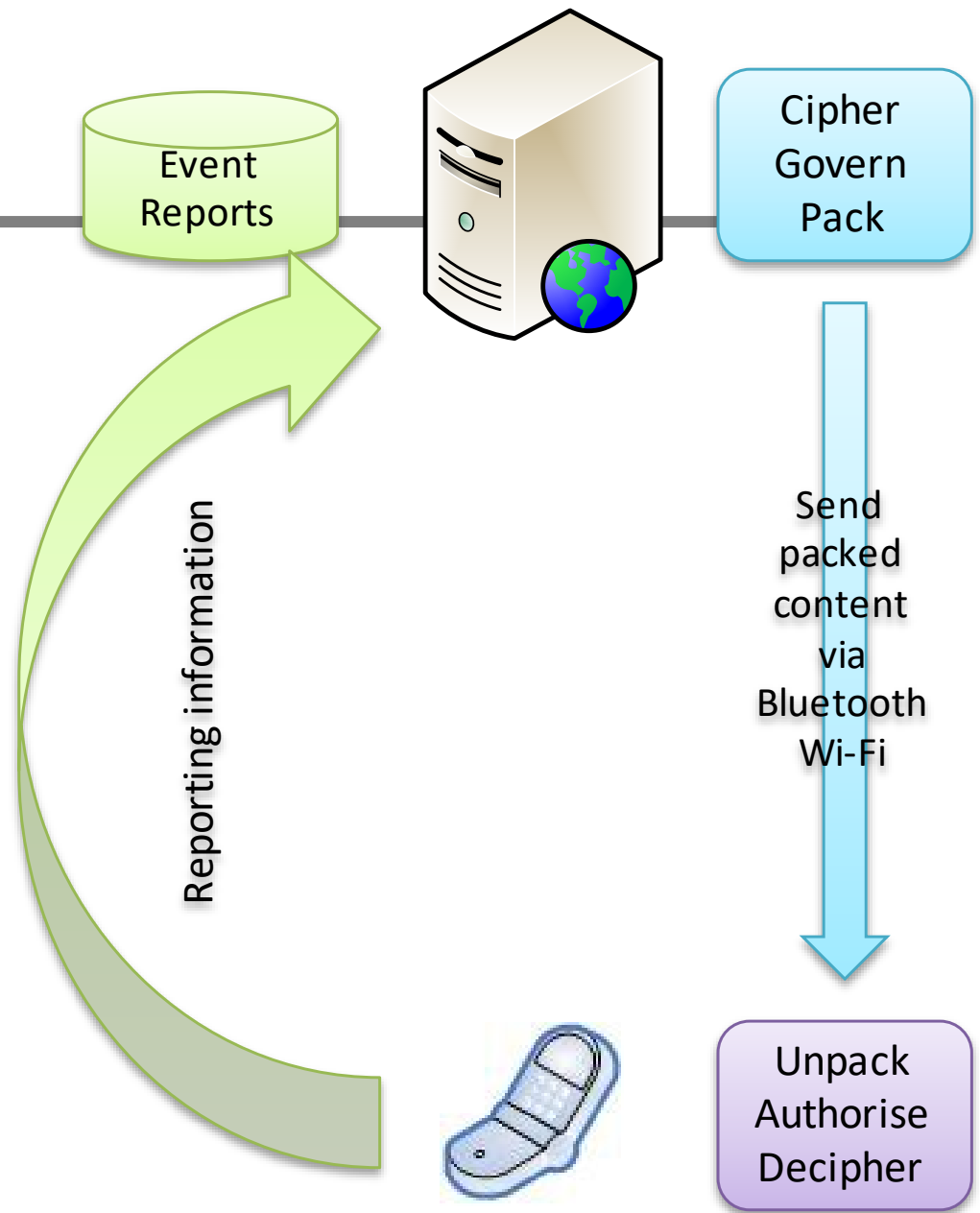


PMS
Server



AXCS

- Event reporting and protected content transmission



Bibliography

- Servlet Tutorial

- <https://www.tutorialspoint.com/servlets/index.htm>

- Jsp Tutorial

- <https://www.tutorialspoint.com/jsp/index.htm>

- WS example

- https://www.w3schools.com/xml/xml_services.asp

- WSDL REST service example

- <https://developer.ibm.com/articles/ws-restful/>

- Reference SOAP - MTOM

- <http://www.redbooks.ibm.com/redpapers/pdfs/redp4884.pdf>

Bibliography

- Java Platform, Enterprise Edition (Java EE) 8, The Java EE Tutorial
 - <https://javaee.github.io/tutorial/toc.html>
- Java EE 8 tutorial, The web tier
 - <https://javaee.github.io/tutorial/partwebtier.html>
- Jakarta EE specifications (9 and beyond)
 - <https://jakarta.ee/specifications/>

Aplicacions Distribuïdes

Silvia Llorente
Jaime Delgado

Distributed Multimedia Applications Group
Departament d'Arquitectura de Computadors

(*) Some slides come from other sources